

UNIVERSIDADE DE SANTIAGO DE
COMPOSTELA



ESCOLA TÉCNICA SUPERIOR DE ENXEÑARÍA

Telaria

**Aplicación móbil colaborativa para a xestión integral
de viaxes en grupo**

Autor:

Jorge Otero Pailos

Titores:

Pedro Gamallo Fernández

Víctor José Gallego Fontenla

Grao en Enxeñaría Informática

Xuño 2026

Traballo de Fin de Grao presentado na Escola Técnica Superior de Enxeñaría da
Universidade de Santiago de Compostela para a obtención do Grao en Enxeñaría
Informática

Agradecementos

Quero expresar o meu sincero agradecemento a todas as persoas que me apoiaron e me axudaron neste proxecto. En primeiro lugar, á universidade, á facultade e aos meus titores por ensinarme todo o necesario pra a realización deste traballo e brindarme esta oportunidade. En segundo lugar, á miña familia, parella e amigos por todo o demais.

Resumo

Este Traballo de Fin de Grao ten como obxectivo o deseño e desenvolvemento de **Telaria**, unha aplicación móbil colaborativa para a xestión integral de viaxes en grupo. A aplicación permite coordinar de forma centralizada todos os aspectos dunha viaxe compartida: o seguimento de gastos e débedas entre os participantes, a planificación de eventos mediante un calendario común, o almacenamento de documentos relevantes, a comunicación entre os membros mediante un chat de grupo e a asistencia dun chatbot conversacional baseado en intelixencia artificial.

A implementación levouse a cabo empregando tecnoloxías modernas para o desenvolvemento de aplicacións móbiles multiplataforma, concretamente **React Native** con **Expo** no frontend e **Spring Boot** no backend, seguindo unha arquitectura *API-first* baseada na especificación **OpenAPI**. A infraestrutura inclúe ademais servizos de almacenamento de ficheiros, modelos de linguaxe locais e despregamento mediante contedores.

Como conclusión, o proxecto ofrece unha solución completa ao problema de coordinación habitual nas viaxes en grupo, integrando nunha única plataforma funcionalidades que adoitan xestionarse de forma dispersa entre distintas ferramentas.

Índice xeral

1. Introducción	1
1.1. Obxectivos xerais	1
1.2. Motivación e contexto	2
1.3. Solucións existentes e diferenciación	2
1.4. Descrición do sistema	3
1.5. Relación da documentación que conforma a memoria	5
1.6. Información adicional de interese	5
2. Especificación de Requisitos	7
2.1. Descrición xeral	7
2.1.1. Perspectiva do produto	7
2.1.2. Funcionalidade do produto	7
2.1.3. Características dos usuarios	8
2.1.4. Restricións	8
2.1.5. Suposicións e dependencias	8
2.2. Interfaces con outros sistemas	8
2.2.1. Interface de usuario	8
2.2.2. Interfaces de hardware	8
2.2.3. Interfaces de software	9
2.2.4. Interfaces de comunicación	9
2.3. Requisitos funcionais	9
2.4. Casos de uso	10
2.5. Requisitos non funcionais	13
2.5.1. Seguridade	13
2.5.2. Privacidade e protección de datos	13
2.5.3. Rendemento	13
2.5.4. Usabilidade	14
2.5.5. Mantibilidade	14
2.5.6. Portabilidade e despregamento	14
2.5.7. Interoperabilidade	14
2.5.8. Fiabilidade	14
2.6. Información que o sistema debe almacenar	14
2.6.1. Almacenamento de ficheiros (MinIO)	15
2.6.2. Información almacenada no dispositivo (frontend)	16

3. Deseño	17
3.1. Arquitectura xeral do sistema	17
3.2. Arquitectura do backend	18
3.3. Modelo de datos	20
3.4. Deseño da API REST	21
3.5. Arquitectura do frontend	22
3.6. Deseño da interface de usuario	23
3.6.1. Linguaxe visual e temas	23
3.6.2. Componentes e patróns de interface	24
3.6.3. Animacións e retroalimentación visual	25
3.6.4. Accesibilidade	25
3.6.5. Análise heurística	27
3.7. Fluxos de comunicación entre componentes	29
3.7.1. Autenticación e renovación de tokens	29
3.7.2. Chat de grupo en tempo real (SSE)	29
3.7.3. Subida de documentos con URL prefirmada	30
3.7.4. Asistente de IA en <i>streaming</i>	30
3.8. Seguridade	31
3.9. Integración con servizos externos	32
3.10. Equipamento hardware e software	32
4. Probas	33
4.1. Estratexia de probas	33
4.2. Probas automatizadas do backend	33
4.3. Probas automatizadas do frontend	34
4.4. Validación funcional do sistema	35
4.5. Avaliación de usabilidade con usuarios reais	36
4.6. Resultados acadados	37
5. Conclusións e Posibles Ampliacións	39
5.1. Grao de cumprimento dos obxectivos	39
5.2. Valoración técnica	40
5.3. Posibles ampliacións	40
A. Manual técnico	41
A.1. Requisitos do entorno	41
A.2. Estrutura do repositorio	41
A.3. Servizos de apoio (Podman)	41
A.4. Execución do backend	42
A.5. Execución do frontend	42
A.6. Contrato OpenAPI e xeración de código	42
A.7. Execución das probas	42
A.8. Configuración e segredos	43
A.9. Integración continua	43

B. Manual de usuario	45
B.1. Instalación e acceso	45
B.2. Rexistro e inicio de sesión	45
B.3. Pantalla principal: as miñas viaxes	46
B.4. Crear e unirse a viaxes	47
B.5. Xestión de membros	48
B.6. Gastos e liquidacións	48
B.7. Eventos e calendario	49
B.8. Documentos compartidos	50
B.9. Chat de grupo	51
B.10. Asistente de intelixencia artificial	51
B.11. Amizades	52
B.12. Invitacións	52
B.13. Perfil	52
B.14. Axustes	53
B.15. Eliminación da conta	54
B.16. Mensaxes de erro comúns	54
C. Licenza	55
Bibliografía	57

Índice de figuras

2.1. Diagrama de casos de uso de Telaria (actor único: Usuario).	11
3.1. Arquitectura xeral e despregamento de Telaria.	18
3.2. Componentes e capas do backend.	19
3.3. Modelo de datos (entidades e relacións principais).	20
3.4. Arquitectura do frontend.	22
3.5. Mapa de navegación da aplicación.	23
3.6. Paleta de cores de Telaria: cores globais e específicas de cada tema.	24
3.7. Pantalla representativa da aplicación (calendario compartido da viaxe).	25
3.8. Comparación dos temas claro e escuro: pantalla de balances da viaxe.	27
3.9. Secuencia de autenticación e renovación do <i>access token</i> .	29
3.10. Secuencia do chat de grupo mediante <i>Server-Sent Events</i> .	30
3.11. Secuencia de subida de documentos con URL prefirmada.	30
3.12. Secuencia do asistente de IA con resposta en <i>streaming</i> .	31
4.1. Evidencia da validación funcional (capturas dos escenarios sobre a aplicación)	36
B.1. Pantallas de rexistro e inicio de sesión	46
B.2. Pantalla principal, coas viaxes do usuario	47
B.3. Creación de viaxe e unión mediante código QR	48
B.4. Membros da viaxe e solicitudes pendentes	48
B.5. Lista de gastos e liquidacións	49
B.6. Calendario e creación de eventos	50
B.7. Listaxe de documentos compartidos	51
B.8. Chat de grupo da viaxe	51
B.9. Asistente de IA da viaxe	51
B.10. Lista de amigos e solicitudes de amizade	52
B.11. Caixa de entrada de invitacións	52
B.12. Perfil de usuario	53
B.13. Pantalla de axustes	54
B.14. Confirmación de eliminación de conta	54

Índice de cadros

1.1. Comparativa de funcionalidades entre Telaria e as alternativas existentes.	3
2.3. Datos persistidos localmente no dispositivo do cliente.	16
3.1. Resumo dos principais recursos da API REST.	21
3.2. Avaliación heurística da interface de Telaria segundo as dez heurísticas de Nielsen.	28
3.3. Equipamento software e hardware do sistema.	32
4.1. Distribución das probas automatizadas do backend.	33
4.2. Cobertura de código do backend (JaCoCo).	34
4.3. Cobertura de código do frontend (Jest) sobre os módulos probados.	35
4.4. Escenarios de validación funcional e a súa trazabilidade cos requisitos.	36
5.1. Grao de cumprimento dos obxectivos.	39

Capítulo 1

Introdución

1.1. Obxectivos xerais

O obxectivo principal deste traballo é o deseño e desenvolvemento de **Telaria**, unha aplicación móbil colaborativa para a xestión integral de viaxes en grupo. A idea de partida é sinxela: ofrecer un *espazo de viaxe* único e compartido no que un grupo de persoas poida organizar toda a vida dunha viaxe (antes, durante e despois) sen ter que saltar entre media ducia de aplicacións xenéricas non pensadas para ese fin.

O problema que motiva o proxecto é a dispersión habitual das ferramentas empregadas durante a planificación e o transcurso dunha viaxe compartida: os gastos xestiónanse nun grupo de mensaxería, os eventos en aplicacións de calendario individuais, os documentos en servizos de almacenamento xenéricos e a comunicación en plataformas non específicas para o contexto da viaxe. Esta fragmentación obriga a cada participante a manter manualmente sincronizada a información espallada e provoca friccións recorrentes: reservas que se perden no historial dun chat, contas que ninguén ten claro quen debe a quen, ou planos que só coñece a persoa que os organizou.

Fronte a esa dispersión, Telaria centraliza nun único sistema os seguintes aspectos:

- Rexistro de gastos compartidos e cálculo automático das liquidacións entre membros, minimizando o número de transferencias necesarias para saldar as débedas.
- Planificación de eventos mediante un calendario común, con soporte para localización xeográfica.
- Almacenamento de documentos relevantes para a viaxe (reservas, billetes, etc.).
- Chat de grupo entre os membros da viaxe.
- Asistente conversacional baseado en intelixencia artificial, con historial persistente por viaxe.

Á marxe destes obxectivos centrais, a aplicación incorpora unha funcionalidade complementaria de carácter social (unha rede de amizades entre usuarios) que non constitúe un módulo principal, senón unha mellora da experiencia de uso. O seu propósito é simplificar as invitacións recorrentes entre usuarios habituais e reforzar a retención, facilitando que as persoas que proban a aplicación sigan empregándoa en sucesivas viaxes.

Como obxectivos técnicos, o proxecto busca aplicar unha arquitectura *API-first* e tecnoloxías multiplataforma modernas, de xeito que a solución sexa funcional tanto en iOS como en Android desde unha única base de código.

1.2. Motivación e contexto

A organización dunha viaxe en grupo é, na práctica, un problema de coordinación entre varias persoas que comparten un mesmo obxectivo pero usan ferramentas distintas e illadas entre si. Durante a fase de planificación adóitanse acumular as reservas de aloxamento e transporte no correo ou na galería de cada quen, os horarios nun calendario persoal e as propostas de actividades nun grupo de mensaxería; xa durante a viaxe, súmanse a ese caos os tickets dos gastos compartidos e as fotos dos documentos importantes. Ningunha desas ferramentas foi concibida para o contexto concreto dunha viaxe, polo que a información queda repartida e desactualizada.

Esta fragmentación traduce-se en custos reais para o grupo. O máis evidente é o económico: ao final da viaxe ninguén sabe con certeza canto puxo cada persoa nin como saldar as contas, e a liquidación faise a man, de forma propensa a erros e a transferencias innecesarias. A iso engádese a perda de información (un billete enterrado nun fío de chat de centos de mensaxes), a dependencia dunha única persoa que «leva» a organización e a dificultade de que todos os participantes teñan unha visión completa e actualizada do plan.

A utilidade que perseguimos con Telaria é precisamente eliminar ese traballo de coordinación manual. Ao reunir gastos, calendario, documentos, comunicación e asistencia intelixente nun mesmo espazo compartido por viaxe, cada participante accede sempre á mesma información actualizada e o esforzo de mantela sincronizada desaparece. O público ao que se dirixe non é o da viaxe corporativa nin o das axencias, senón os grupos informais (cuadrillas de amizades, familias ou compañeiros) que viaxan xuntos de cando en vez e que hoxe resolven esta coordinación cun conxunto disperso de aplicacións xenéricas.

1.3. Solucións existentes e diferenciación

No mercado existen aplicacións que cobren *partes* do problema descrito, pero ningunha o aborda de forma integrada para o contexto dunha viaxe en grupo. As alternativas máis representativas son as seguintes:

- **Tricount** [17]: especializada na xestión de gastos compartidos e no cálculo de quen debe a quen. Resolve ben a parte económica, pero non ofrece calendario, almacenamento de documentos, chat de grupo nin asistencia intelixente. Foi, de feito, unha das inspiracións do proxecto: o seu enfoque da liquidación de gastos é o punto de partida que Telaria estende ao resto de necesidades dunha viaxe.
- **WhatsApp e grupos de mensaxería** [18]: cobren a comunicación do grupo, que é onde acaba converxendo o resto da organización por falta dun lugar mellor. Todo o demais (gastos, calendario, documentos) queda como información non estruturada dentro do fío de conversa, sen ningún tratamento específico.

- **Google Calendar** [19]: resolve a planificación temporal, pero é unha ferramenta de calendario xenérica e centrada no individuo, non un espazo colaborativo organizado por viaxe nin integrado co resto de necesidades.
- **Wanderlog e TravelPerk** [20, 21]: son planificadores de viaxe máis completos, pero orientados respectivamente ao descubrimento de itinerarios e á xestión de viaxes corporativas. Teñen un soporte feble da liquidación de gastos compartidos entre iguais e carecen dun asistente de intelixencia artificial integrado.

A diferenza destas opcións, Telaria intégraas todas nun mesmo produto e engade catro trazos que a distinguen: **(1)** un *espazo de viaxe único* que reúne nun só lugar gastos, calendario, documentos, chat e asistente; **(2)** un *modelo plenamente colaborativo sen roles*, no que todos os membros teñen as mesmas capacidades e ningún depende dun «organizador»; **(3)** a *liquidación automática con minimización de débedas*, que reduce ao mínimo o número de transferencias necesarias; e **(4)** un *asistente de intelixencia artificial integrado por viaxe*, que ningunha das alternativas ofrece. A táboa 1.1 resume esta comparación.

Funcionalidade	Tricount	WhatsApp	G. Calendar	Wanderlog	Telaria
Gastos e liquidación	✓	✗	✗	parcial	✓
Calendario / eventos	✗	✗	✓	✓	✓
Documentos compartidos	✗	parcial	✗	parcial	✓
Chat de grupo	✗	✓	✗	✗	✓
Asistente de IA	✗	✗	✗	✗	✓
Modelo colaborativo	parcial	parcial	✗	✓	✓

Cadro 1.1: Comparativa de funcionalidades entre Telaria e as alternativas existentes.

1.4. Descrición do sistema

Telaria é unha aplicación móbil orientada a grupos de persoas que realizan viaxes conxuntas. Calquera usuario pode rexistrarse na plataforma e, a partir de aí, crear novas viaxes ou unirse a viaxes existentes mediante invitación, solicitude ou código QR.

Dentro de cada viaxe, todos os membros teñen acceso completo e en igualdade de condicións ás funcionalidades da aplicación. Non existe distinción de roles: o sistema está deseñado baixo un modelo plenamente colaborativo no que calquera participante pode rexistrar gastos, crear eventos, subir documentos, enviar mensaxes ao chat e interactuar co asistente de IA. A continuación descríbense, desde o punto de vista do usuario, as principais funcionalidades que compoñen este espazo de viaxe.

Ciclo de vida da viaxe. A viaxe é a unidade central arredor da que se organiza todo o demais. Un usuario crea unha viaxe e convida a outras persoas, que poden incorporarse por invitación directa, solicitando o acceso ou escaneando un código QR; calquera membro pode abandonala e o creador xestionar as solicitudes pendentes.

Gastos e liquidacións. Cada membro pode rexistrar os gastos que paga, indicando os beneficiarios e unha categoría (aloxamento, transporte, comida, etc.). A partir deses rexistros, o módulo calcula automaticamente o balance de cada persoa e propón un

conxunto de liquidacións aplicando un algoritmo de minimización de débedas, de xeito que o número de transferencias necesarias para saldar todos os saldos sexa o mínimo posible. As liquidacións xa realizadas conservan-se nun historial.

Eventos e calendario. O grupo constrúe un itinerario común sobre un calendario compartido. Cada evento pode ter data, hora, duración e unha localización xeográfica, que se pode escoller buscando un enderezo ou seleccionándoo directamente nun mapa interactivo con xeocodificación inversa.

Documentos compartidos. A viaxe dispón dun repositorio de documentos relevantes (reservas, billetes, fotos, etc.) que calquera membro pode subir, descargar ou eliminar. Para os documentos de imaxe xérase de forma asíncrona unha miniatura de previsualización, de modo que a listaxe se mostre con axilidade sen descargar os ficheiros orixinais.

Chat de grupo. Cada viaxe inclúe unha sala de conversa propia para a coordinación entre os membros, con entrega de mensaxes en tempo real e aviso de mensaxes sen ler.

Asistente de intelixencia artificial. Cada viaxe conta cun asistente conversacional que axuda na planificación e responde consultas. Mantén un historial de conversa independente por viaxe, de modo que o contexto se preserve entre sesións.

Rede de amizades e xestión de conta. De maneira complementaria, a plataforma ofrece unha rede de amizades entre usuarios: calquera usuario pode engadir outros como amigos e, a partir de aí, convidalos ás súas viaxes de forma máis áxil que mediante o identificador ou o código QR. Esta funcionalidade non altera o modelo colaborativo descrito, senón que actúa como atallo para os usuarios habituais. Cada usuario xestiona ademais o seu propio perfil (nome, correo, avatar e contrasinal) e pode eliminar a súa conta, operación que require a confirmación do contrasinal. A aplicación está dispoñible en tres idiomas (galego, español e inglés) e ofrece temas de aparencia claro e escuro.

1.5. Relación da documentación que conforma a memoria

Sección	Contido
Resumo	Breve resumo das principais contribucións do traballo.
Introdución	Obxectivos xerais, descrición do sistema e tecnoloxías e arquitecturas empregadas.
Especificación de Requisitos	Requisitos funcionais e non funcionais, interfaces con outros sistemas e información que o sistema debe almacenar.
Deseño	Arquitectura do sistema, división en compoñentes e comunicación entre eles. Equipamento hardware e software empregado.
Probas	Plan de probas con evidencias que verifican a funcionalidade e corrección global do sistema. Resultados acadados.
Conclusións e Posibles Ampliacións	Grao de cumprimento dos obxectivos e posibles vías de mellora futura.
Bibliografía	Lista completa de referencias citadas no texto.
Manuais Técnicos	Documentación para desenvolvedores: código fonte, recursos necesarios e operacións de modificación e proba.
Manuais de Usuario	Documentación autocontida para usuarios finais: instalación, uso, configuración e mensaxes de erro.

1.6. Información adicional de interese

Arquitectura API-first con OpenAPI

O deseño do sistema parte da especificación da API antes de calquera implementación. Isto establece un contrato explícito e verificable entre frontend e backend, permite xerar documentación interactiva de forma automática (Swagger UI) [6] e facilita o desenvolvemento en paralelo de ambos os extremos.

React Native e Expo

O frontend está desenvolvido con React Native [1], o que permite compartir a mesma base de código para iOS e Android. Expo [2] engade sobre React Native unha capa de utilidades que simplifica a xestión de dependencias nativas, o proceso de compilación e a distribución da aplicación, reducindo a complexidade de configuración do proxecto.

Spring Boot

O backend emprega Spring Boot [4] pola súa madurez e polo seu ecosistema integrado: Spring MVC para a capa REST, Spring Data JPA para a persistencia, Spring Security para a autenticación e autorización, e integración directa con bibliotecas de xeración de documentación OpenAPI.

Autenticación baseada en JWT

O mecanismo de autenticación utiliza dous tokens complementarios. O *access token*, de tipo JWT [8] e vida curta, inclúese en cada petición á API e permite verificar a identidade do usuario sen consultar a base de datos. O *refresh token*, de tipo opaco e almacenado no servidor, empregase exclusivamente para renovar o *access token* e pode invalidarse explicitamente no peche de sesión.

Ollama

O asistente conversacional funciona sobre Ollama [10], un motor de inferencia local de modelos de linguaxe grande. A execución local elimina a dependencia de servizos externos de pago, preserva a privacidade dos datos dos usuarios e ofrece latencias predecibles independentes de terceiros.

Almacenamento de obxectos compatible con S3

Os documentos compartidos almacénanse nun servizo compatible co protocolo S3 [11]. A elección deste estándar garante a independencia do provedor e aproveita o soporte nativo dispoñible en Spring Boot mediante o AWS SDK.

PostgreSQL

A base de datos relacional escollida é PostgreSQL [7], pola súa robustez, polo seu soporte nativo para UUID e tipos de data con fuso horario, e pola súa excelente integración con Spring Data JPA.

Podman

O despregamento dos servizos realízase mediante contedores OCI xestionados con Podman [13]. A diferenza de Docker, Podman opera sen un daemon centralizado e permite executar os contedores sen privilexios de superusuario (*rootless*), o que mellora o illamento de seguridade do entorno de execución.

Tarefas periódicas de limpeza

Para os documentos de imaxe xérase unha miniatura de previsualización reducida que se mostra na listaxe de documentos sen necesidade de descargar o ficheiro orixinal. Esta xeración realízase de forma asíncrona (fóra do fluxo de subida e da ruta de petición) mediante unha tarefa programada que procesa periodicamente os documentos aínda sen miniatura. Deste xeito a subida e a listaxe de documentos manteñen unha latencia baixa e independente do custo de procesamento de imaxe.

Capítulo 2

Especificación de Requisitos

A especificación de requisitos segue a estrutura do estándar **IEEE 830** para documentos de especificación de requisitos software (SRS), adaptada ás necesidades deste proxecto: unha descrición xeral do produto, as interfaces con outros sistemas, os requisitos funcionais e non funcionais, os casos de uso máis relevantes e a información que o sistema debe almacenar.

2.1. Descrición xeral

2.1.1. Perspectiva do produto

Telaria é unha aplicación móbil baseada nunha arquitectura cliente–servidor. O frontend, desenvolvido con React Native e Expo, comunícase cun backend Spring Boot mediante unha API REST documentada con OpenAPI. O backend integra á súa vez servizos externos: un motor de inferencia de linguaxe (Ollama) para o asistente de IA e un almacén de obxectos compatible con S3 para os documentos compartidos.

2.1.2. Funcionalidade do produto

O sistema cobre os seguintes módulos funcionais:

- Autenticación e xestión de conta de usuario
- Creación e adhesión a viaxes, xestión de membros
- Rexistro de gastos compartidos e cálculo de liquidacións
- Planificación de eventos con localización xeográfica
- Almacenamento de documentos compartidos
- Chat de grupo entre membros da viaxe
- Asistente conversacional de intelixencia artificial por viaxe

Ademais dos módulos anteriores, o sistema inclúe dúas funcionalidades complementarias: unha rede de amizades entre usuarios (funcionalidade de calidade de vida orientada a axilizar as invitacións recorrentes) e a xestión da propia conta, incluída a súa eliminación con confirmación de contrasinal.

2.1.3. Características dos usuarios

Tipo	Descrición
Usuario	Persoa rexistrada na aplicación. Pode crear ou unirse a viaxes e, dentro delas, ten acceso completo a todas as funcionalidades de forma colaborativa e en igualdade de condicións co resto de membros.

2.1.4. Restricións

- A aplicación é multiplataforma (iOS e Android) e funciona sobre Expo.
- Toda comunicación co servidor realízase mediante HTTP e API REST.
- O sistema require conexión á rede para a totalidade das súas operacións.

2.1.5. Suposicións e dependencias

- O servizo Ollama está dispoñible e accesible no servidor para o módulo de IA.
- O servizo de almacenamento PostgreSQL está operativo e accesible para a persistencia de datos.
- Existe un servizo de almacenamento compatible co protocolo S3 para os documentos.
- Os requisitos aquí descritos son estables durante o desenvolvemento.

2.2. Interfaces con outros sistemas

2.2.1. Interface de usuario

Interface gráfica móbil nativa (React Native) con navegación en pestanas e pilas de pantallas, adaptada a iOS e Android sen necesidade de compilación separada por plataforma.

2.2.2. Interfaces de hardware

- Dispositivo móbil do usuario con sistema operativo iOS ou Android e conexión á rede.
- Servidor con capacidade para executar contedores Podman.

2.2.3. Interfaces de software

Sistema	Descrición
API REST propia	Interface principal entre frontend e backend; documentada con OpenAPI/Swagger e consumida mediante JSON sobre HTTP.
Ollama	Servizo local de inferencia de modelos de linguaxe grande, accedido por HTTP para alimentar o asistente conversacional.
Almacenamento S3	Servizo de almacenamento de obxectos (compatible con S3) para a subida e descarga de documentos compartidos.
PostgreSQL	Base de datos relacional para a persistencia de todas as entidades do sistema.

2.2.4. Interfaces de comunicación

O frontend e o backend comunícanse mediante HTTP. A autenticación baséase en dous tokens: un *access token* JWT de vida curta incluído en cada petición, e un *refresh token* opaco de vida longa almacenado no servidor e utilizado para renovar o *access token* sen necesidade de volver a autenticarse.

2.3. Requisitos funcionais

ID	Nome		Descrición	Prior.
RF01	Rexistro de usuario		O sistema permite crear unha conta con nome de usuario, correo electrónico e contrasinal.	Alta
RF02	Autenticación		O sistema autentica usuarios mediante un <i>access token</i> JWT; o <i>refresh token</i> opaco inválidase no peche de sesión.	Alta
RF03	Crear viaxe		Un usuario autenticado pode crear novas viaxes.	Alta
RF04	Unirse a viaxe		Un usuario pode unirse a unha viaxe mediante invitación recibida, solicitude de unión ou código QR.	Alta
RF05	Xestión de membros		Calquera membro pode invitar outros usuarios á viaxe e xestionar as solicitudes de unión pendentes.	Alta
RF06	Xestión de gastos		Calquera membro pode rexistrar un gasto (importe, nome, categoría, usuario pagador e conxunto de beneficiarios), consultalo, filtralo por categoría ou nome e eliminalo.	Alta
RF07	Liquidacións		O sistema calcula automaticamente as liquidacións entre membros, minimizando o número de transferencias necesarias para saldar as débedas; ademais, calquera membro pode rexistrar liquidacións xa realizadas e consultar o histórico de liquidacións da viaxe.	Alta

ID	Nome	Descrición	Prior.
RF08	Xestión de eventos	Calquera membro pode crear, consultar e eliminar eventos con nome, data e hora de inicio, duración e localización xeográfica (nome, enderezo, coordenadas e URL de mapa).	Alta
RF09	Documentos compartidos	Calquera membro pode subir, consultar e eliminar documentos asociados á viaxe; os documentos de imaxe mostran unha miniatura de previsualización xerada automaticamente.	Media
RF10	Chat de grupo	Os membros dunha viaxe poden enviar e consultar mensaxes de texto no chat común da viaxe.	Media
RF11	Asistente de IA	Cada viaxe dispón dun chatbot conversacional con historial persistente entre sesións.	Baixa
RF12	Rede de amizades	Funcionalidade complementaria: un usuario pode enviar solicitudes de amizade a outro (polo seu identificador ou código QR), aceptalas ou rexeitalas, consultar a súa lista de amigos e eliminar amizades.	Baixa
RF13	Convidar amigos á viaxe	Funcionalidade complementaria que permite convidar amigos a unha viaxe directamente desde a lista de amizades, como atallo ás invitacións por identificador ou QR.	Baixa
RF14	Eliminación de conta	Un usuario pode eliminar a súa propia conta; a operación require a confirmación do contrasinal e elimina os datos persoais asociados.	Baixa
RF15	Xestión do perfil	Un usuario pode consultar e actualizar o seu nome de usuario e correo electrónico (o cambio de correo require a confirmación do contrasinal), cambiar o contrasinal e subir ou consultar o seu avatar.	Media
RF16	Saír dunha viaxe	Calquera membro pode abandonar unha viaxe; se é o último membro, a viaxe e todos os seus datos asociados elimínanse automaticamente.	Media
RF17	Preferencias da aplicación	O usuario pode personalizar a interface: seleccionar o tema visual (claro ou escuro), o idioma (galego, castelán ou inglés) e activar o modo de aforro de datos.	Baixa

2.4. Casos de uso

A partir dos requisitos funcionais identifícanse os casos de uso máis relevantes do sistema. Dado que non existe distinción de roles, o único actor é o **Usuario** rexistrado, que interactúa con todas as funcionalidades. A Figura 2.1 resume os casos de uso agrupados por área funcional, e a continuación detállanse os fluxos principais.

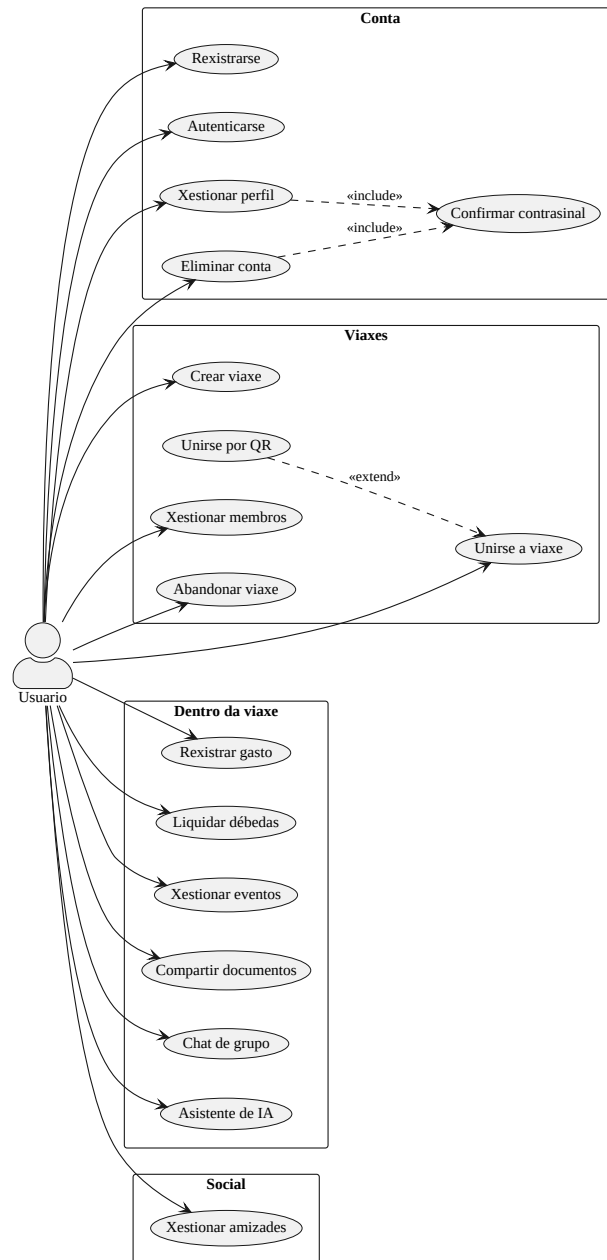


Figura 2.1: Diagrama de casos de uso de Telaria (actor único: Usuario).

CU01. Rexistro e autenticación.

- **Actor:** Usuario.
- **Precondicións:** dispón da aplicación instalada e de conexión á rede.
- **Fluxo principal:** (1) introduce nome de usuario, correo e contrasinal e solicita o rexistro; (2) o sistema crea a conta, almacena o contrasinal cifrado e devolve un *access token* JWT e un *refresh token*; (3) o usuario queda autenticado. O inicio de sesión posterior realízase con correo e contrasinal.
- **Fluxos alternativos:** correo xa rexistrado → o sistema rexeita o rexistro; credenciais incorrectas no inicio de sesión → erro de autenticación; caducidade do *access token* → o cliente renóvao de forma transparente co *refresh token*.
- **Poscondición:** o usuario dispón dunha sesión activa.

- **Requisitos:** RF01, RF02.

CU02. Unirse a unha viaxe.

- **Actor:** Usuario.
- **Precondicións:** usuario autenticado.
- **Fluxo principal:** (1) obtén unha invitación, coñece o identificador ou escanea o código QR dunha viaxe; (2) o sistema rexistra a operación correspondente; (3) o usuario pasa a ser membro da viaxe.
- **Fluxos alternativos:** por invitación (un membro convídao e o usuario acéptaa); por solicitude (o usuario solicita unirse e calquera membro a aproba); por QR (o escaneo resólvese nunha das vías anteriores).
- **Poscondición:** o usuario figura na lista de membros da viaxe.
- **Requisitos:** RF04, RF05.

CU03. Rexistrar un gasto e liquidar débedas.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) rexistra un gasto (importe, nome, categoría, pagador e beneficiarios); (2) o sistema garda o gasto e recalcula os balances; (3) o membro consulta quen debe a quen; (4) cando se salda unha débeda, rexístrase a liquidación correspondente.
- **Fluxos alternativos:** o pagador ou algún beneficiario non é membro da viaxe → erro de validación.
- **Poscondición:** os balances reflicten o novo gasto; as liquidacións rexistradas reducen as débedas pendentes.
- **Requisitos:** RF06, RF07.

CU04. Xestionar os membros da viaxe.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) convida a outro usuario (por identificador, amizade ou QR) ou consulta as solicitudes de unión pendentes; (2) acepta ou rexeita unha solicitude; (3) o sistema actualiza a lista de membros.
- **Fluxos alternativos:** usuario xa membro ou xa convidado → conflito.
- **Poscondición:** a composición da viaxe queda actualizada. Calquera membro pode realizar estas accións: non hai roles.
- **Requisitos:** RF05, RF13.

CU05. Subir un documento.

- **Actor:** Usuario (membro da viaxe).
- **Precondicións:** o usuario é membro da viaxe.
- **Fluxo principal:** (1) inicia a subida indicando nome e tipo de contido; (2) o sistema crea o rexistro e devolve unha URL prefirmada; (3) o cliente sobe o ficheiro directamente ao almacén de obxectos; (4) o cliente confirma a subida e o sistema marca o documento como dispoñible, xerando unha miniatura de forma asíncrona se é unha imaxe.

- **Fluxos alternativos:** se a subida non se confirma, unha tarefa programada elimina o rexistro orfo.
- **Poscondición:** o documento queda listado e descargable mediante URL prefirrada.
- **Requisitos:** RF09.

CU06. Eliminar a conta.

- **Actor:** Usuario.
- **Precondicións:** usuario autenticado.
- **Fluxo principal:** (1) solicita eliminar a súa conta; (2) o sistema solicita a confirmación do contrasinal; (3) tras a verificación, elimínanse os datos persoais e os artefactos asociados (avatars, documentos) e retírase o usuario das viaxes compartidas; (4) péchase a sesión.
- **Fluxos alternativos:** contrasinal incorrecto → cancélase a operación.
- **Poscondición:** a conta e os datos persoais quedan eliminados.
- **Requisitos:** RF14, RF15.

2.5. Requisitos non funcionais

2.5.1. Seguridade

- **RNF-SE-01.** Control de acceso: só os membros dunha viaxe acceden aos seus datos; calquera usuario autenticado pode crear viaxes.
- **RNF-SE-02.** Os contrasinais almacénanse con hash BCrypt; os *access tokens* JWT teñen vida curta e os *refresh tokens* opacos almacénanse no servidor para limitar a exposición ante compromisos.
- **RNF-SE-03.** As operacións sensíbles sobre a conta (o cambio de correo electrónico e a eliminación da conta) requiren a confirmación do contrasinal actual.

2.5.2. Privacidade e protección de datos

- **RNF-PR-01.** A eliminación da conta borra os datos persoais do usuario e os artefactos asociados no almacén de obxectos (avatar e documentos), e retíraos das viaxes compartidas.
- **RNF-PR-02.** As mensaxes de chat (de grupo e do asistente de IA) elimínanse automaticamente tras un período de retención de 30 días mediante unha tarefa programada.

2.5.3. Rendemento

- **RNF-RE-01.** O cálculo de liquidacións realízase no servidor, evitando sobrecargar o dispositivo cliente.
- **RNF-RE-02.** A entrega de mensaxes do chat e das respostas do asistente de IA realízase en tempo real mediante eventos enviados polo servidor (SSE).

- **RNF-RE-03.** A xeración de miniaturas dos documentos de imaxe execútase de forma asíncrona, fóra da ruta de petición, para non penalizar a latencia da subida nin da listaxe.

2.5.4. Usabilidade

- **RNF-US-01.** A interface debe ser intuitiva e usable sen formación previa.
- **RNF-US-02.** A aplicación estará dispoñible en galego, castelán e inglés.

2.5.5. Mantibilidade

- **RNF-MA-01.** O contrato OpenAPI é a fonte de verdade da API: a partir del xéranse automaticamente as interfaces e DTO do backend e os tipos do frontend, garantindo a sincronía entre ambos os extremos.
- **RNF-MA-02.** A lóxica do backend está cuberta por unha batería de probas automatizadas (unitarias e de integración) que facilita a evolución segura do código.

2.5.6. Portabilidade e despregamento

- **RNF-PO-01.** O backend e os servizos auxiliares despreganse en contedores Podman.
- **RNF-PO-02.** A aplicación móbil é compatible con iOS e Android mediante Expo/React Native.

2.5.7. Interoperabilidade

- **RNF-IN-01.** O almacenamento de documentos baséase no protocolo estándar S3, o que garante a independencia respecto dun provedor concreto.

2.5.8. Fiabilidade

- **RNF-FI-01.** O sistema valida os datos de entrada na API e devolve mensaxes de erro claras e estruturadas ao cliente.
- **RNF-FI-02.** Unha tarefa programada reconcilia periodicamente as subidas de documentos non confirmadas (orfas): confirma as que efectivamente existen no almacén de obxectos e elimina os rexistros sen ficheiro asociado, evitando entradas inconsistentes na base de datos.

2.6. Información que o sistema debe almacenar

A persistencia do sistema repártese en tres niveis complementarios segundo a natureza do dato. A información estruturada do dominio (usuarios, viaxes, eventos, gastos, mensaxes, etc.) almacénase nunha base de datos relacional **PostgreSQL**, onde Hibernate mapea cada entidade á súa táboa. Os **ficheiros binarios** (documentos compartidos

e avatares) non se gardan na base de datos, senón nun **almacén de obxectos**. Por último, o dispositivo do usuario conserva localmente un pequeno conxunto de datos de sesión e de preferencias que se detalla máis adiante. A Táboa seguinte recolle as principais entidades persistidas no servidor e os seus atributos.

Entidade	Atributos principais
Usuario	Identificador (UUID), nome de usuario, correo electrónico, contrasinal (hash).
Viaxe	Identificador (UUID), nome, conxunto de membros (usuarios).
Evento	Identificador, nome, data e hora de inicio (con fuso horario), duración en minutos, localización embebida (nome, enderezo, latitude, lonxitude, URL de mapa).
Gasto	Identificador, importe (decimal), nome, categoría (enumeración), usuario pagador, usuario creador do rexistro, conxunto de beneficiarios, marca temporal.
Liquidación	Identificador, importe (decimal), usuario pagador, usuario receptor, marca temporal.
Documento compartido	Identificador, nome de ficheiro, clave no almacén de obxectos, clave da miniatura de previsualización, estado de subida (booleano), usuario creador, marca temporal.
Invitación / Solicitud	Identificador, usuario, viaxe asociada, marca temporal de creación.
Amizade	Identificador (UUID), dous usuarios asociados (almacenados de forma normalizada para garantir a unicidade da parella), marca temporal de creación.
Solicitud de amizade	Identificador (UUID), usuario emisor, usuario receptor, marca temporal de creación.
Mensaxe de chat de grupo	Identificador, contido (texto), autor, viaxe asociada, marca temporal.
Mensaxe de asistente de IA	Identificador, contido (texto), rol (USUARIO ou ASISTENTE), usuario, viaxe asociada, marca temporal.
Token de refresco	Token (identificador), identificador do usuario asociado.

2.6.1. Almacenamento de ficheiros (MinIO)

Os ficheiros que sobe o usuario (documentos compartidos das viaxes e imaxes de avatar) non son axeitados para gardarse nunha base de datos relacional: trátase de obxectos binarios potencialmente grandes que cargarían as táboas, dificultarían as copias de seguranza e penalizarían o rendemento das consultas. Por iso o sistema delega a súa custodia nun **almacén de obxectos compatible coa API de Amazon S3**, para o que se emprega **MinIO**, unha solución de código aberto. A vantaxe de MinIO é que é **autoxestionado**: en lugar de contratar un servizo de almacenamento na nube como Amazon S3 (onde os ficheiros residen na infraestrutura dun provedor externo e o seu uso se factura por consumo), MinIO execútase na propia infraestrutura do proxecto, despregándose de xeito sinxelo como un contedor (Docker / Podman) xunto co resto de

servizos. Isto elimina a dependencia e os custos dun provedor comercial e, ao expoñer a mesma API que S3, permitiría migrar a ese servizo no futuro sen apenas cambios no código.

A base de datos non garda o contido do ficheiro, senón unicamente os **metadatos** necesarios para localizalo: a clave do obxecto dentro do almacén (*object key*), a clave da miniatura de previsualización e o estado da subida. Deste xeito mantense unha única fonte de verdade (o rexistro relacional referencia o obxecto físico) e evítase a duplicación de datos pesados.

A interacción co almacén realízase mediante **URLs prefirmadas**: cando un usuario quere subir ou descargar un ficheiro, o backend xera unha URL temporal e asinada que autoriza esa operación concreta durante un período limitado. O cliente comunícase **directamente** con MinIO empregando esa URL, polo que o servidor de aplicación nunca chega a recibir nin a servir os bytes do ficheiro. Esta arquitectura descarga o backend, mellora a escalabilidade e reduce a latencia das transferencias.

2.6.2. Información almacenada no dispositivo (frontend)

Ademais da persistencia no servidor, a aplicación móbil garda localmente no dispositivo un conxunto reducido de información para soste a sesión entre execucións e recordar as preferencias da persoa usuaria. As credenciais de sesión (os *tokens*) gárdanse no **almacén seguro e cifrado do sistema operativo**, respaldado pola *keychain* de iOS ou o *keystore* de Android, mentres que o resto de valores, ao non ser sensibles, se persisten no almacenamento clave-valor convencional do dispositivo. A Táboa 2.3 resume o que se garda no cliente.

Cadro 2.3: Datos persistidos localmente no dispositivo do cliente.

Clave	Contido e propósito
<code>refreshToken</code>	Token de refresco da sesión (cifrado); permite obter novos tokens de acceso sen volver iniciar sesión.
<code>accessToken</code>	Token de acceso vixente (cifrado) que autoriza as peticións á API.
<code>userEmail</code>	Correo electrónico da persoa usuaria autenticada.
<code>username</code>	Nome de usuario mostrado na interface.
<code>app_language</code>	Idioma escollido para a interface (es / en / gl).
<code>app_theme</code>	Tema visual seleccionado (claro / escuro).
<code>app_data_saver</code>	Indicador do modo de aforro de datos.
<code>lastTripId</code>	Identificador da última viaxe aberta, para restaurar a navegación.
<code>lastTripTab</code>	Última pestana visitada dentro dunha viaxe.

Este almacenamento local é deliberadamente mínimo: non se replica no dispositivo o contido das viaxes nin datos doutros usuarios, senón unicamente o estritamente preciso para reanudar a sesión e personalizar a experiencia. Ao pechar sesión, tanto as credenciais cifradas como o estado de navegación bórranse do dispositivo.

Capítulo 3

Deseño

Neste capítulo descríbese como se constrúe o sistema: a súa arquitectura xeral, a división en compoñentes e a comunicación entre eles, o modelo de datos, o deseño da API e dos extremos cliente e servidor, os fluxos máis relevantes e, por último, o equipamento hardware e software empregado. Sempre que é posible recórrese a representacións gráficas para facilitar a comprensión da estrutura do produto.

3.1. Arquitectura xeral do sistema

Telaria segue unha arquitectura **cliente–servidor** de tres capas lóxicas (presentación, lóxica de negocio e persistencia) e adopta un enfoque *API-first*: o contrato da API, especificado con OpenAPI, é o punto de partida do desenvolvemento e o elemento que vincula o cliente co servidor. A Figura 3.1 mostra a visión de conxunto.

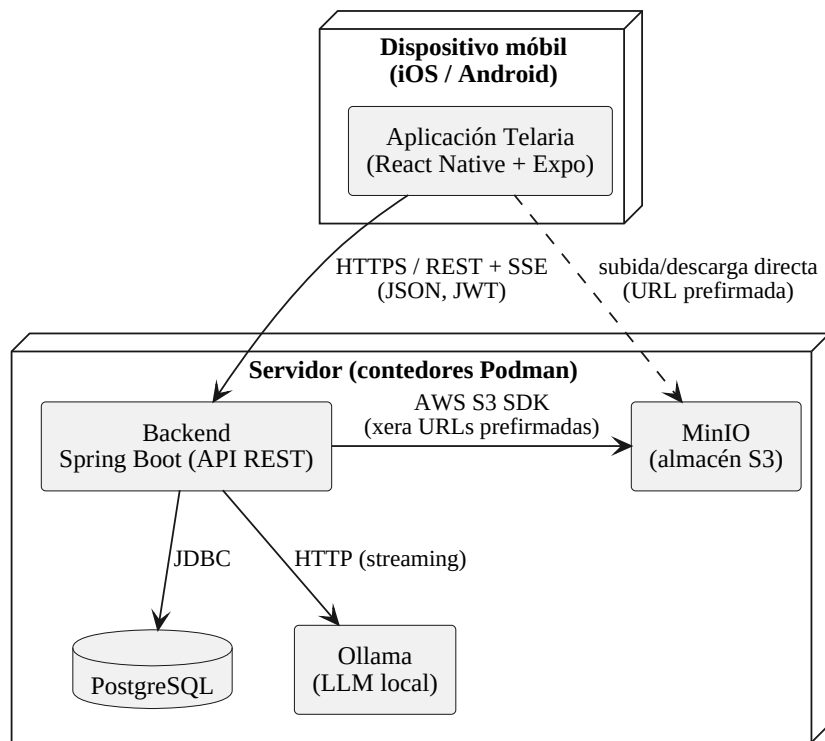


Figura 3.1: Arquitectura xeral e despregamento de Telaria.

O **cliente** é unha aplicación móbil multiplataforma (React Native + Expo) que se executa en iOS e Android. O **servidor** agrupa, en contedores xestionados con Podman, o backend Spring Boot e os tres servizos de apoio: a base de datos PostgreSQL, o almacén de obxectos compatible con S3 (MinIO) e o motor de inferencia de modelos de linguaxe (Ollama).

A comunicación principal entre cliente e servidor realízase sobre HTTP mediante unha API REST que intercambia JSON e protexe cada petición cun *access token* JWT. Para as funcionalidades en tempo real (chat de grupo e asistente de IA), o servidor emprega *Server-Sent Events* (SSE), que permiten un fluxo unidireccional de eventos do servidor cara ao cliente sobre a mesma conexión HTTP. Finalmente, a subida e a descarga de documentos non pasan polo backend: este limítase a xerar *URLs prefirmadas* co que o cliente fala **directamente** co almacén de obxectos, descargando o servidor da transferencia de ficheiros.

3.2. Arquitectura do backend

O backend organízase nunha arquitectura por capas que separa claramente as responsabilidades, como se aprecia na Figura 3.2.

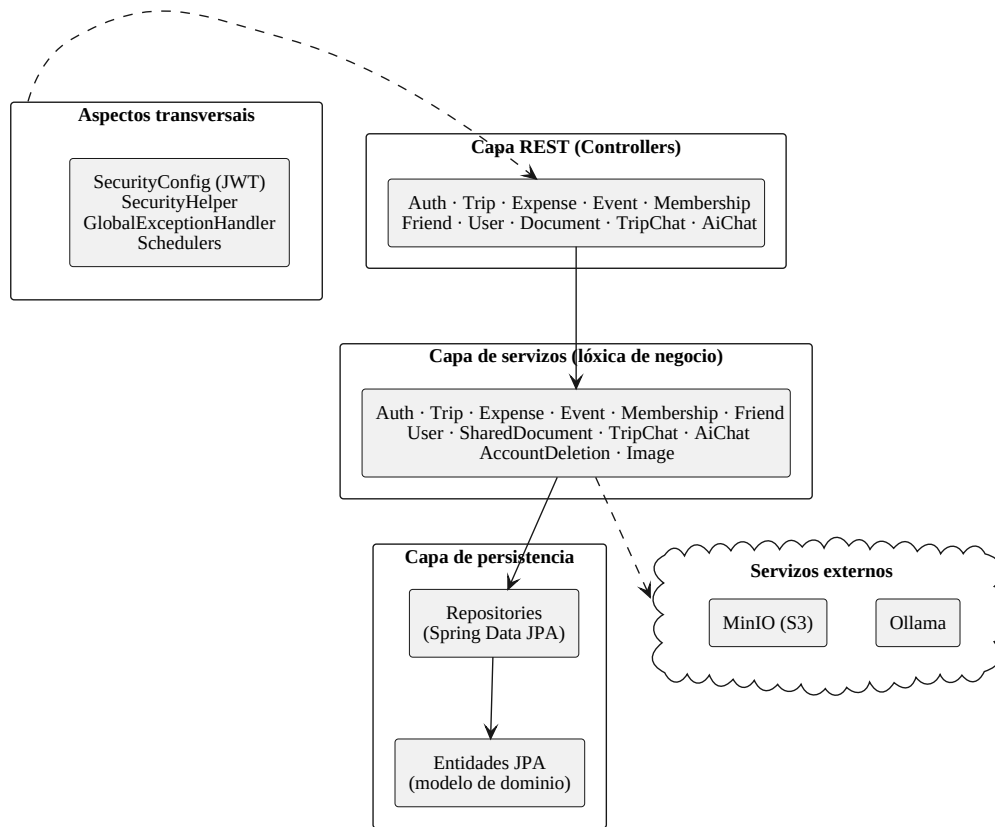


Figura 3.2: Compoñentes e capas do backend.

- **Capa REST (controllers).** Expón os *endpoints* da API. As interfaces destes controladores xéranse automaticamente a partir do contrato OpenAPI, e as clases que as implementan conteñen unicamente a tradución entre a petición HTTP e a chamada ao servizo correspondente.
- **Capa de servizos.** Concentra a lóxica de negocio: validacións, cálculo de liquidacións, resolución de invitacións e solicitudes, xestión de ficheiros, etc. É a única capa que coordina varias entidades e repositorios dentro dunha mesma transacción.
- **Capa de persistencia.** Componse dos repositorios de Spring Data JPA e das entidades do modelo de dominio, que Hibernate mapea ás táboas de PostgreSQL.
- **Aspectos transversais.** A configuración de seguridade (validación de tokens JWT con Spring Security), a utilidade de extracción do usuario autenticado (`SecurityHelper`), o manexo centralizado de erros (`GlobalExceptionHandler`, que devolve respostas no formato *Problem Details*, RFC 7807) [9] e as tarefas programadas (*schedulers*) de limpeza e xeración de miniaturas.

A autorización aplícase na capa de servizos: o acceso aos datos dunha viaxe verifícase comprobando que o usuario autenticado pertence aos seus membros, de modo que ningún usuario poida acceder á información de viaxes das que non forma parte.

3.3. Modelo de datos

A Figura 3.3 representa o modelo de dominio coas entidades persistentes e as súas relacións.

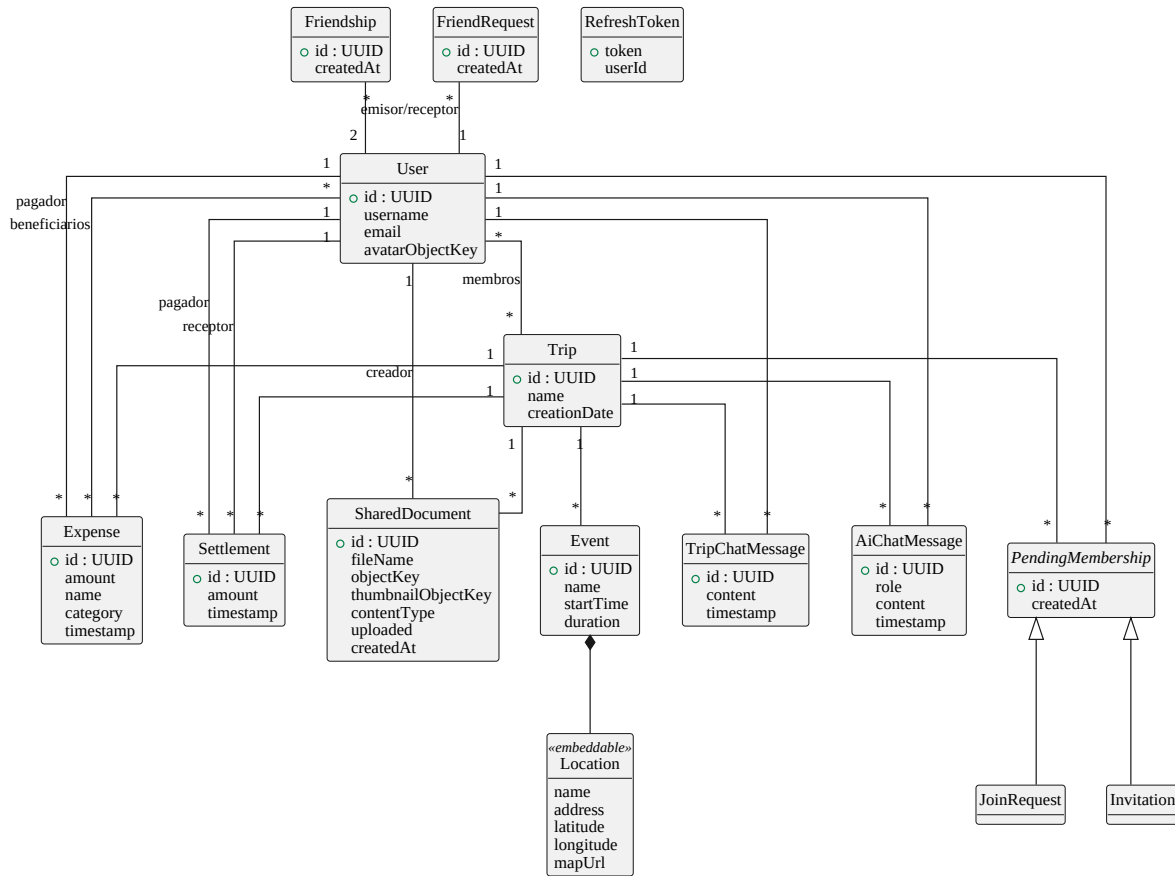


Figura 3.3: Modelo de datos (entidades e relacións principais).

As decisións de deseño máis salientables son:

- **Identificadores UUID.** Todas as entidades principais empregan UUID como clave primaria, o que evita expor identificadores secuenciais e facilita a xeración de claves no lado da aplicación.
- **Relacións varios-a-varios.** A pertenza ás viaxes (**User** – **Trip**) e os beneficiarios dun gasto (**Expense** – **User**) módelanse mediante táboas de unión.
- **Normalización das amizades.** Cada **Friendship** almacena os dous usuarios de forma ordenada (o de identificador menor primeiro), o que garante a unicidade da parella e simplifica as consultas bidireccionais.
- **Valor embebido.** A localización dun evento (**Location**: nome, enderezo, coordenadas e URL de mapa) módelase como un obxecto embebido dentro de **Event**, ao non ter identidade propia (considérase que os diferentes eventos non compartirán ubicación frecuentemente).

- **Xerarquía de membros pendentes.** `Invitation` e `JoinRequest` comparten estrutura (viaxe, usuario e marca temporal) a través da superclase `PendingMembership`, diferenciándose só na dirección da petición.
- **Token de refresco persistente.** O `RefreshToken` almacénase no servidor para poder invalidalo no peche de sesión.

O esquema da base de datos non se xestiona con ferramentas de migración: emprégase a xeración automática de Hibernate (`ddl-auto`), apropiada para o contexto deste proxecto. Na sección de conclusións recóllese a adopción de migracións explícitas como posible mellora de cara a un entorno de produción.

3.4. Deseño da API REST

A API é o contrato central do sistema e especificase con **OpenAPI** de forma modular: un documento principal (`api.yaml`) referencia, mediante `$ref`, ficheiros independentes para as rutas (`paths/`) e para os esquemas de datos (`schemas/`). Esta organización mantén o contrato lexible malia o seu tamaño.

A partir deste contrato xérase código automaticamente en ambos os extremos:

- No **backend**, o complemento *OpenAPI Generator* produce as interfaces dos controladores e os obxectos de transferencia de datos (DTO).
- No **frontend**, a ferramenta `openapi-typescript` xera os tipos TypeScript que describen as peticións e respostas.

Deste xeito, calquera cambio no contrato propágase automaticamente como tipos a ambos os extremos, e calquera incoherencia detéctase en tempo de compilación. Ademais, a partir da mesma especificación ofrécese documentación interactiva da API (Swagger UI). A Táboa 3.1 resume os principais recursos da API.

Recurso	Responsabilidade
<code>/auth</code>	Rexistro, inicio e peche de sesión, renovación de tokens.
<code>/users</code>	Perfil, contrasinal, avatar, eliminación de conta.
<code>/trips</code>	Creación e consulta de viaxes.
<code>/trips/[tripID]/expenses</code>	Gastos, balances e liquidacións.
<code>/trips/[tripID]/events</code>	Eventos do calendario.
<code>/trips/[tripID]/documents</code>	Documentos compartidos (subida en varios pasos).
<code>/trips/[tripID]/group-chat</code>	Chat de grupo (consulta, envío e fluxo SSE).
<code>/trips/[tripID]/ai-chat</code>	Asistente de IA (historial e resposta en <i>streaming</i>).
Invitacións, solicitudes, amizades	Xestión de membros e da rede social de usuarios.

Cadro 3.1: Resumo dos principais recursos da API REST.

3.5. Arquitectura do frontend

O cliente organízase tamén por capas, como amosa a Figura 3.4. A navegación baséase en **Expo Router**, un sistema de rutas baseado na estrutura de ficheiros, que agrupa as pantallas en tres grupos ((**auth**), (**main**) e (**trip**)).

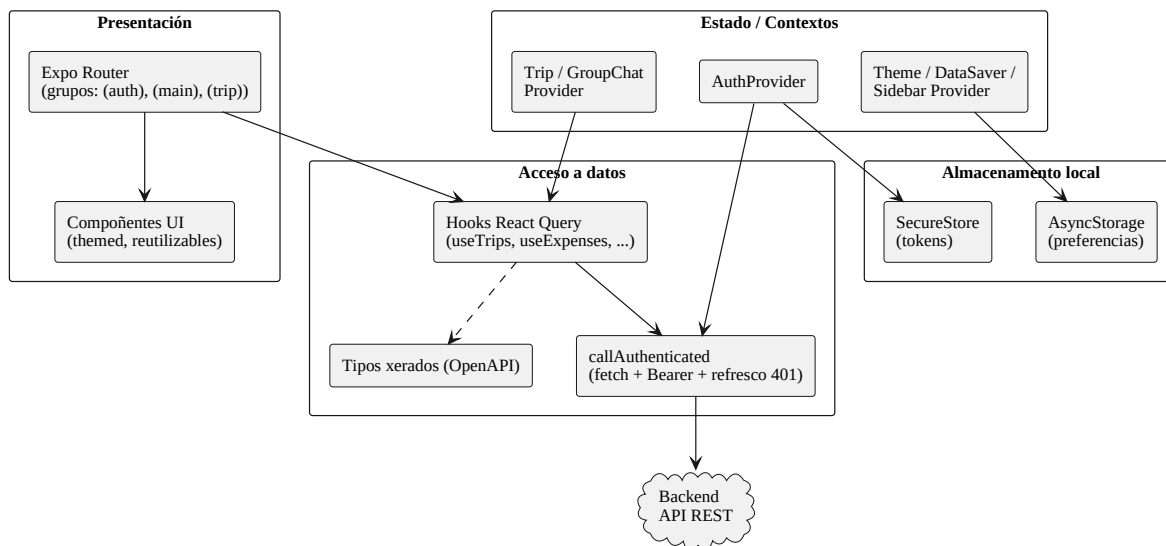


Figura 3.4: Arquitectura do frontend.

O estado da aplicación divídese en dous tipos. O **estado de servidor** (datos que proveñen do backend) xestiónase con **React Query** [3], que se encarga da caché, da revalidación e da sincronización; cada recurso ten os seus *hooks* de consulta e mutación e unha factoría de claves de caché. O **estado de aplicación** (sesión, tema, viaxe actual, chat) maniféstase mediante contextos de React (**AuthProvider**, **ThemeProvider**, **TripProvider**, **GroupChatProvider**, etc.).

Toda chamada autenticada pasa por unha función envoltorio (**callAuthenticated**) que engade o *access token* á cabeceira **Authorization** e, ante unha resposta 401, renova o token de forma transparente e reintenta a petición. Os tokens gárdanse de forma cifrada (**SecureStore**) e as preferencias do usuario en almacenamento local (**AsyncStorage**). A Figura 3.5 mostra o mapa completo de navegación.

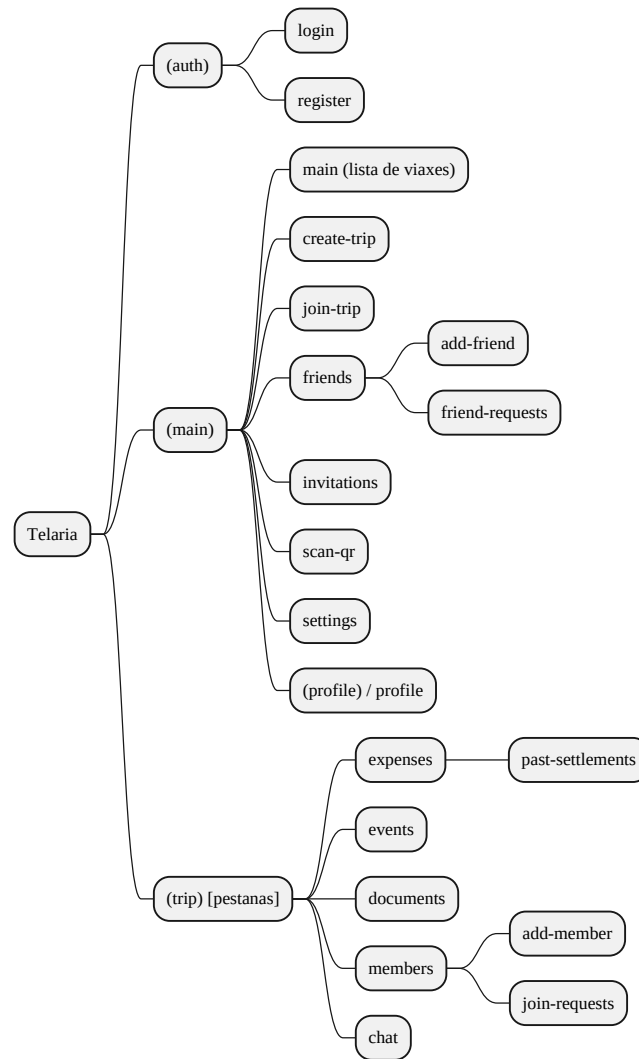


Figura 3.5: Mapa de navegación da aplicación.

3.6. Deseño da interface de usuario

Alén da súa arquitectura técnica, o frontend conta cunha linguaxe visual propia e co-
idada. Nesta sección descríbese o sistema visual, os compoñentes e patróns de interface,
as animacións e a accesibilidade.

3.6.1. Linguaxe visual e temas

A identidade visual de Telaria articúlase arredor dunha paleta monocromática cen-
trada no violeta. A aplicación ofrece dous temas (claro e escuro) que o usuario selecciona
manualmente e que se conservan entre sesións (non segue automaticamente o axuste do
sistema). A Figura 3.6 recolle as cores principais.



Figura 3.6: Paleta de cores de Telaria: cores globais e específicas de cada tema.

A tipografía emprega as fontes do sistema (sen fontes propias) e a xerarquía visual conséguese mediante o peso e o espazado entre letras, máis que mediante familias tipográficas distintas. A estética complementábase con esquinas redondeadas e brillos sutís (*glow*) de ton violeta arredor dos elementos destacados.

3.6.2. Componentes e patróns de interface

A interface constrúese sobre un conxunto de componentes reutilizables con tema: `ThemedText`, `ThemedButton` e `ThemedInput` encapsulan o estilo base; `UserAvatar` mostra a imaxe do usuario ou, na súa ausencia, as iniciais, e respecta o modo de aforro de datos; e `SegmentedControl` ofrece un selector con animación. O cromo de navegación componse dunha barra de pestanas inferior que resalta a pestana activa cun brillo e amosa badges de notificación, e dun menú lateral que se presenta como panel deslizante inferior (*bottom sheet*). O asistente de IA ábrese nun modal a pantalla completa accesible desde un botón flotante. Completan o repertorio as tarxetas, os modais e os diálogos de confirmación. A Figura 3.7 mostra unha pantalla representativa.

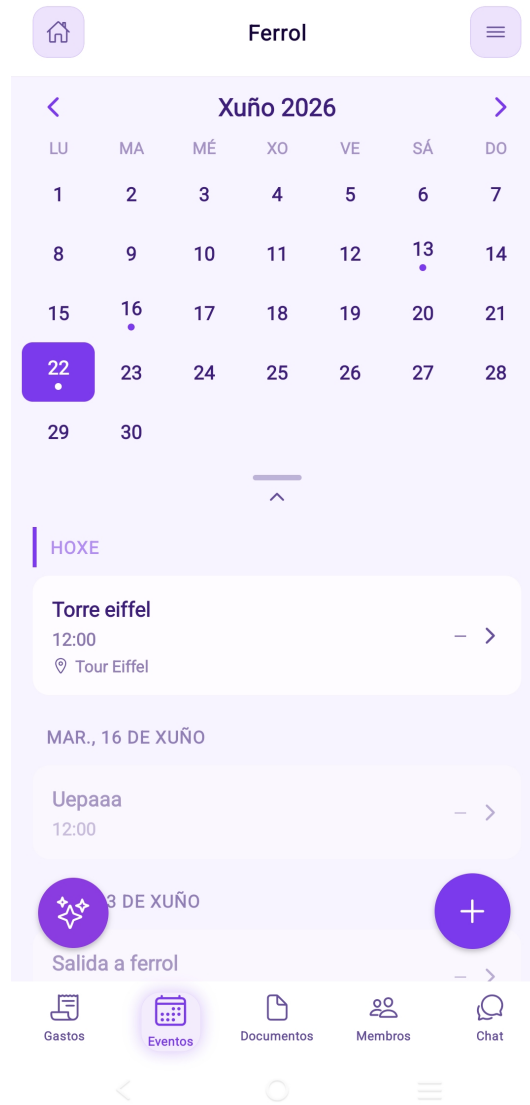


Figura 3.7: Pantalla representativa da aplicación (calendario compartido da viaxe).

3.6.3. Animacións e retroalimentación visual

As animacións baséanse na API `Animated` de React Native. A retroalimentación de pulsación aplica unha lixeira redución de escala e de opacidade aos elementos interactivos; o `SegmentedControl` desprázase cun efecto de resorte; o escáner de códigos QR mostra unha liña de varrido e un pulso; o contido reaxústase de forma animada á altura do teclado; e o fondo das pantallas inclúe unha textura sutil (grella de puntos e manchas de cor ambientais) que achega profundidade sen distraer.

3.6.4. Accesibilidade

No tocante á accesibilidade, a aplicación adopta varias medidas, aínda que tamén deixa marxe de mellora que se describe con transparencia.

Medidas adoptadas. A aplicación está internacionalizada en galego, castelán e inglés (cf. RNF-US-02), o que mellora a comprensión para usuarios de distintas linguas. O dobre tema claro/escuro axuda ás persoas sensibles á luz ou con baixa visión, e o modo de aforro de datos reduce a carga de imaxes. Ademais, os controis interactivos de toda a aplicación, a barra de pestanas (que expón a pestana seleccionada mediante `accessibilityState`), os menús, o asistente de IA e os botóns de acción das distintas pantallas (engadir, eliminar, navegar, etc.), incorporan etiquetas (`accessibilityLabel`) e roles (`accessibilityRole`) de accesibilidade internacionalizados, de xeito que os lectores de pantalla anuncian cada control co seu nome; a retroalimentación de pulsación é consistente en toda a interface. Finalmente, a paleta de cores axustouse para que as combinacións de texto e fondo cumpran o limiar de contraste WCAG AA. Ademais, respéctase a preferencia de movemento reducido do sistema (as animacións desactívanse ou acúrtanse cando o usuario a activa) e as compoñentes de texto compartidas limitan o factor máximo de escala da tipografía (`maxFontSizeMultiplier`) para que os tamaños grandes non rompan os deseños. Os controis interactivos amplían ademais a súa área de pulsación (`hitSlop`) ata o tamaño mínimo recomendado, e as accións consecuentes (saír da viaxe, eliminar a conta, pechar a sesión, etc.) inclúen pistas (`accessibilityHint`) que describen o seu efecto.

Limitacións. O límite de escala da tipografía aplícase ás compoñentes de texto compartidas e aos controis principais, pero aínda non a todos os textos da interface.

Traballo futuro. Unha mellora adicional incluíría estender o límite de escala da tipografía a todos os textos da interface (e non só ás compoñentes compartidas). Esta revisión pode apoiarse nas comprobacións de accesibilidade da ferramenta react-doctor, xa presente no proxecto. A Figura 3.8 ilustra os dous temas.

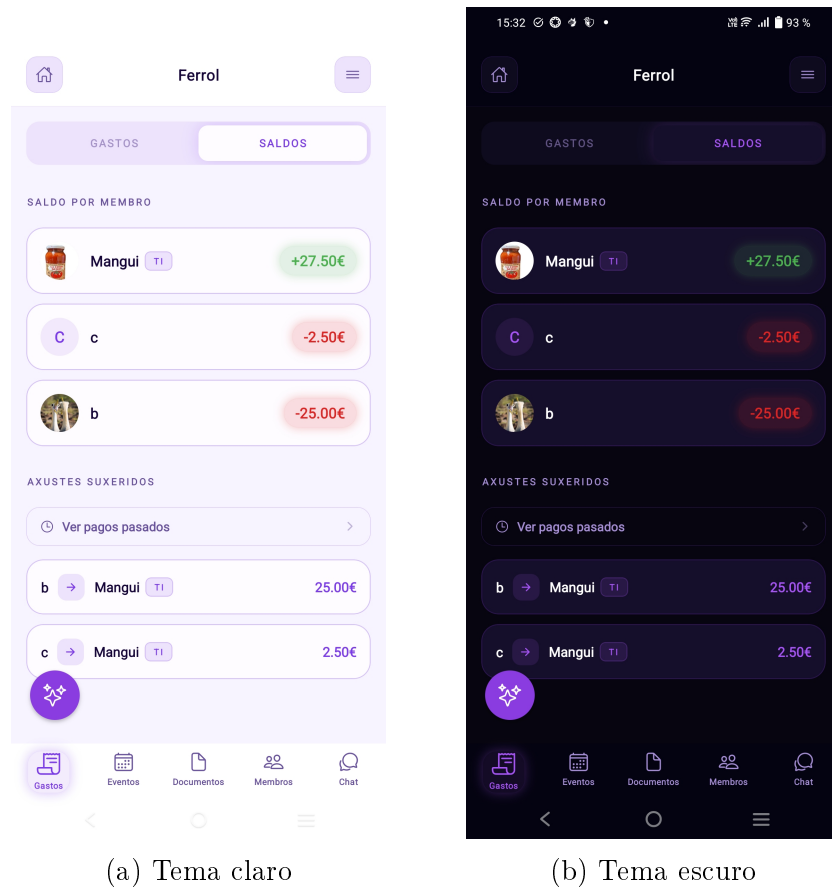


Figura 3.8: Comparación dos temas claro e escuro: pantalla de balances da viaxe.

3.6.5. Análise heurística

Co obxectivo de identificar posibles problemas de usabilidade desde unha perspectiva experta, a interface de Telaria avalíouse segundo as **dez heurísticas de usabilidade de Nielsen** [16]. Esta avaliación complementa as probas de pensamento en voz alta con usuarios reais descritas no capítulo 4. A Táboa 3.2 resume como o deseño aborda cada heurística.

Heurística	Aplicación en Telaria
Visibilidade do estado do sistema	Indicadores de carga nas chamadas á API, progresión visual na subida de documentos, actualizacións en tempo real do chat e do asistente de IA (<i>streaming</i>) e <i>badges</i> de notificación na barra de pestanas.
Correspondencia co mundo real	Terminoloxía de viaxes familiar (gastos, membros, eventos, calendario), categorías de gasto naturais e mapa interactivo para seleccionar localizacións.
Control e liberdade do usuario	Navegación cara atrás en todos os niveis, posibilidade de abandonar unha viaxe e de eliminar a conta, e cancelación de calquera modal sen consecuencias.
Consistencia e estándares	Compoñentes con tema reutilizables (<code>ThemedText</code> , <code>ThemedButton</code> , <code>ThemedInput</code>), paleta e tipoloxía uniformes e patróns de navegación estándar para plataformas móbiles (barra de pestanas, <i>bottom sheet</i>).
Prevención de erros	Validación de formularios antes do envío, diálogos de confirmación para operacións destrutivas (eliminar conta, saír da viaxe) e tipado estrito de extremo a extremo a través do contrato OpenAPI.
Recoñecemento antes que recordo	Barra de pestanas sempre visible, historial de mensaxes e de gastos accesible en todo momento e información contextual da viaxe activa presente en cada pantalla.
Flexibilidade e eficiencia	Unión a viaxes por código QR, convites directos a amigos e acceso inmediato aos documentos do día seleccionado no calendario.
Deseño estético e minimalista	Estética limpa, ausencia de información superflua en pantalla e textura de fondo sutil que non distrae do contido principal.
Axuda para recuperarse de erros	Mensaxes de erro claras nas operacións fallidas, formato <i>Problem Details</i> (RFC 7807) [9] nas respostas de erro da API e retroalimentación visual ante fallos de rede.
Axuda e documentación	Asistente de IA integrado que coñece o contexto da viaxe activa (eventos, gastos e historial de chat) e pode responder consultas do usuario en linguaxe natural.

Cadro 3.2: Avaliación heurística da interface de Telaria segundo as dez heurísticas de Nielsen.

3.7. Fluxos de comunicación entre compoñentes

3.7.1. Autenticación e renovación de tokens

O inicio de sesión devolve un *access token* JWT de vida curta e un *refresh token* opaco. Cando o *access token* caduca, o cliente detéctao ante unha resposta 401 e renóvao de forma transparente, sen requirir unha nova autenticación do usuario (Figura 3.9).

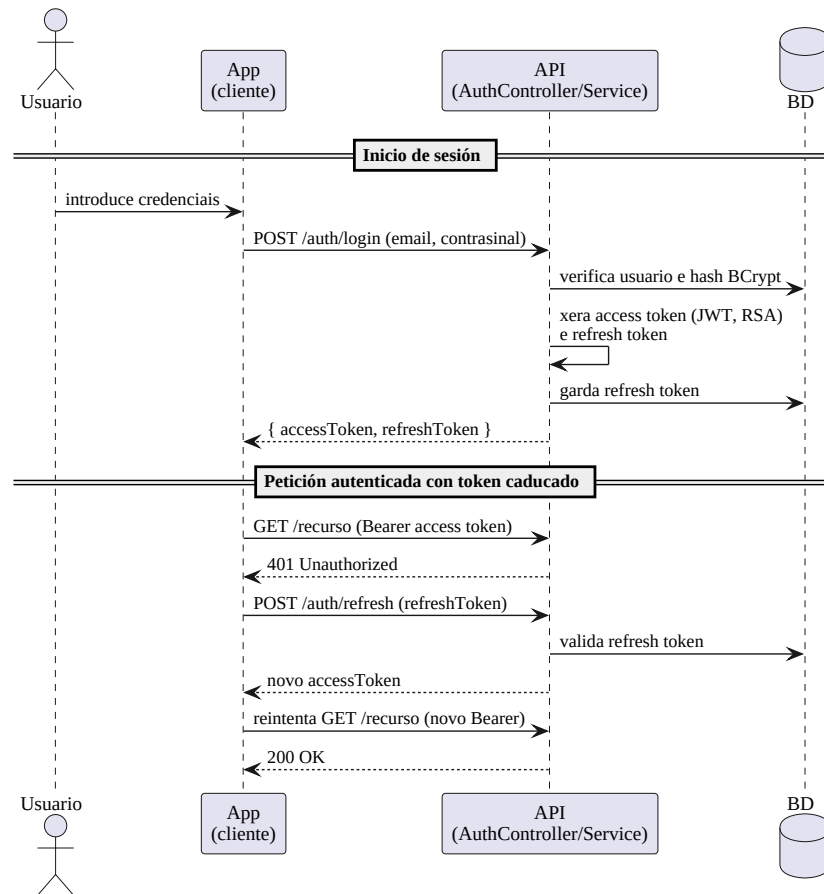


Figura 3.9: Secuencia de autenticación e renovación do *access token*.

3.7.2. Chat de grupo en tempo real (SSE)

Cando un membro abre o chat, o cliente subscríbese a un fluxo SSE. Ao enviar outro membro unha mensaxe, o servidor persístea e difúndea de inmediato a todos os subscritores conectados (Figura 3.10).

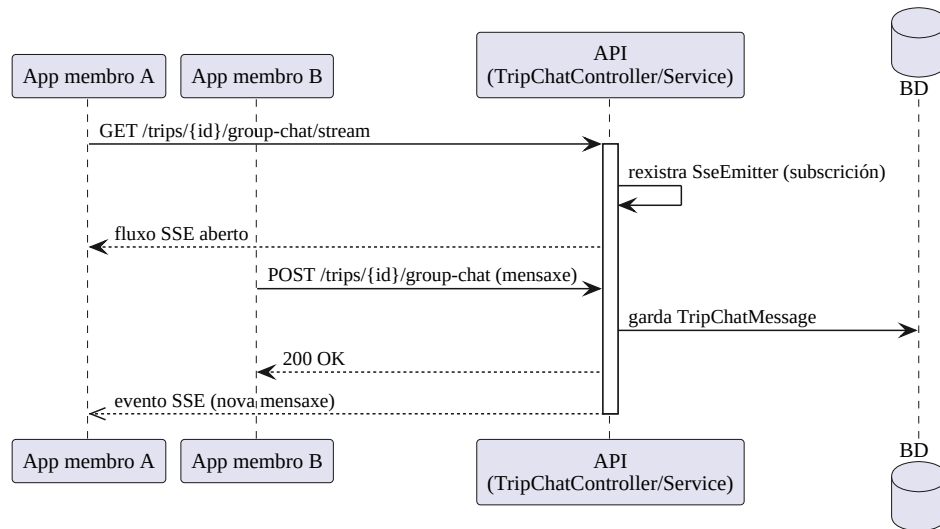


Figura 3.10: Secuencia do chat de grupo mediante *Server-Sent Events*.

3.7.3. Subida de documentos con URL prefirmada

A subida realízase en varios pasos para que o ficheiro non atravesase o backend: este crea o rexistro e devolve unha URL prefirmada, o cliente sobe o ficheiro directamente ao almacén e, finalmente, confirma a subida; se o ficheiro é unha imaxe, xérase unha miniatura de forma asíncrona (Figura 3.11).

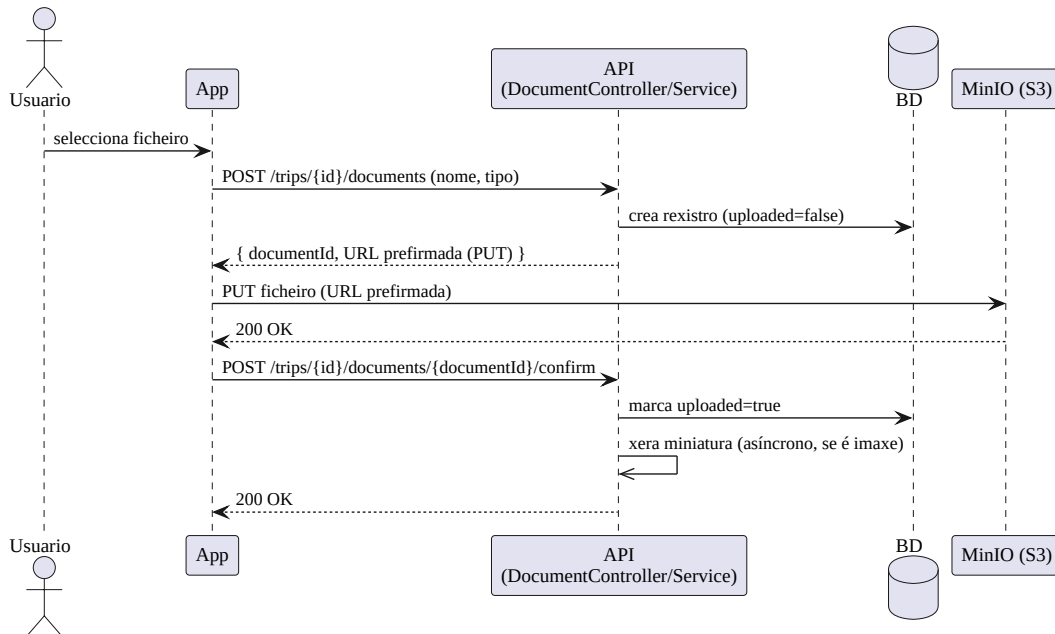


Figura 3.11: Secuencia de subida de documentos con URL prefirmada.

3.7.4. Asistente de IA en *streaming*

Ao recibir unha consulta, o servidor constrúe o contexto da conversa a partir dos eventos e gastos da viaxe e das últimas mensaxes, e remíttello ao modelo local (Olla-

ma). A resposta devólvese token a token cara ao cliente mediante SSE, ofrecendo unha experiencia de escritura progresiva (Figura 3.12).

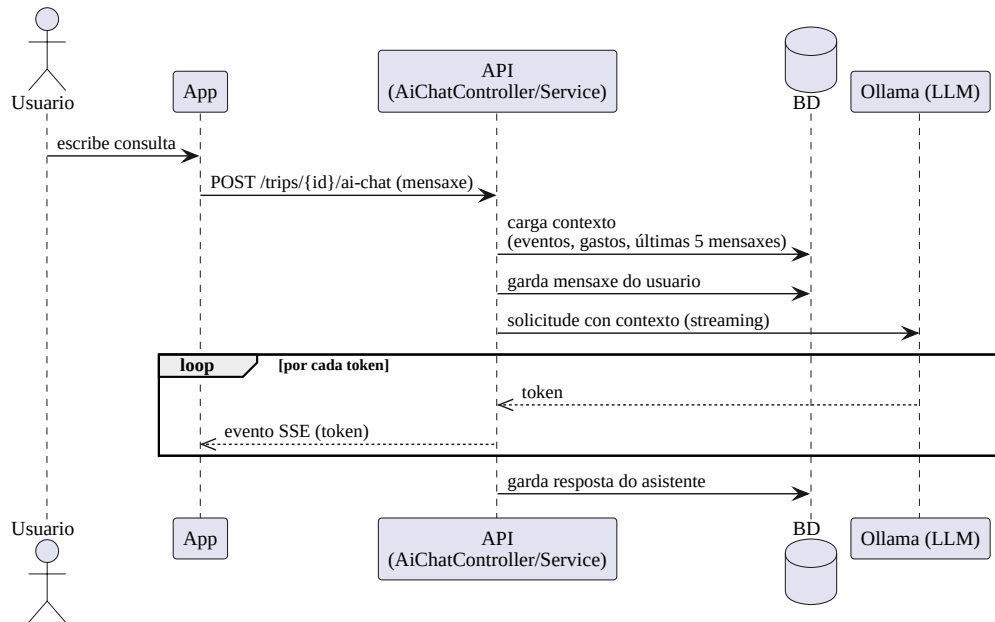


Figura 3.12: Secuencia do asistente de IA con resposta en *streaming*.

3.8. Seguridade

A seguridade do sistema descansa sobre os seguintes piares de deseño:

- **Autenticación con JWT asinado.** No inicio de sesión, o backend asina os seus propios *access tokens* (JWT) cunha clave privada RSA almacenada nun *keystore*; en cada petición valida a sinatura do token coa clave pública, sen necesidade de consultar a base de datos. Esta validación de tokens portadores (*bearer*) apóiase no soporte de JWT de Spring Security [5]. Non se implementa o protocolo OAuth 2.0: non existe un servidor de autorización externo nin fluxos de delegación, senón un esquema de autenticación propio baseado en tokens autoasinaos.
- **Sesións sen estado.** O servidor non mantén sesión; cada petición é autosuficiente grazas ao token, o que mellora a escalabilidade.
- **Contrasinais protexidos.** Os contrasinais almacénanse cifrados con BCrypt [12].
- **Autorización por pertenza.** O acceso aos recursos dunha viaxe está condicionado a que o usuario sexa membro dela.
- **Confirmación en operacións sensíbles.** O cambio de correo electrónico e a eliminación de conta requiren reintroducir o contrasinal actual.

3.9. Integración con servizos externos

- **Ollama.** O asistente de IA delega a inferencia nun modelo de linguaxe executado localmente, ao que o backend accede por HTTP en modo *streaming*.
- **Almacén de obxectos (MinIO/S3).** A xestión de ficheiros realízase co AWS SDK; o backend nunca recibe nin serve os ficheiros, senón que xera URLs prefir-madas de subida e descarga cunha validez limitada.
- **Procesamento de imaxes.** As miniaturas de previsualización e o redimensio-namento de avatares prodúcense reducindo a imaxe a un tamaño máximo e con-vertíndoa a JPEG, sempre fóra da ruta de petición.

3.10. Equipamento hardware e software

O desenvolvemento e a execución do sistema requiren o seguinte equipamento, re-sumido na Táboa 3.3. Dado que ningunha destas tecnoloxías constituía un requisito previo, a súa elección xustifícase nas seccións anteriores e na introdución.

Ámbito	Tecnoloxías e ferramentas
Frontend	React Native, Expo, TypeScript, React Query; ferramentas de Node.js.
Backend	Java, Spring Boot, Spring Data JPA, Spring Security; constru-ción con Gradle.
Persistencia	PostgreSQL.
Almacenamento	MinIO (compatible con S3).
Intelixencia artificial	Ollama (modelo de linguaxe local).
Despregamento	Contedores OCI xestionados con Podman.
Cliente final	Dispositivo móbil iOS ou Android con conexión á rede.

Cadro 3.3: Equipamento software e hardware do sistema.

Capítulo 4

Probas

A verificación da funcionalidade e da corrección global do sistema aborda dúas dimensións complementarias: as **probas automatizadas** do backend, que garanten a corrección técnica da lóxica e da API, e a **validación funcional** de extremo a extremo da aplicación, que comproba que o produto se comporta como espera o usuario.

4.1. Estratexia de probas

A estratexia segue o modelo da pirámide de probas, con tres niveis:

- **Probas unitarias:** illan a lóxica de negocio de cada servizo ou módulo, substituíndo as dependencias por dobres de proba (*mocks*). Aplícanse tanto no backend (JUnit 5) como no frontend (Jest).
- **Probas de integración (E2E):** exercitan a API completa contra unha base de datos e un almacén de obxectos reais, levantados en contedores efémeros.
- **Validación de sistema:** comprobación manual dos fluxos de usuario completos sobre a aplicación en execución.

4.2. Probas automatizadas do backend

O backend conta cunha batería de **410 probas automatizadas** repartidas en 30 clases, que se executan co *framework* JUnit 5. A Táboa 4.1 resume a súa distribución.

Tipo	Clases	Descrición
Unitarias de servizos	11	Lóxica de negocio illada con Mockito.
Integración (E2E)	12	API completa con base de datos e almacén reais (Testcontainers).
Modelo	4	Lóxica propia das entidades de dominio.
Tarefas programadas	3	Comportamento dos <i>schedulers</i> de limpeza.
Total	30	410 probas

Cadro 4.1: Distribución das probas automatizadas do backend.

As probas de integración empregan **Testcontainers** [14], que levanta contedores efémeros de PostgreSQL e MinIO para cada execución, de xeito que as probas se realizan contra servizos reais e non contra simulacións. Unha clase base común (**BaseE2ETest**) orquestra o arrinque dos contedores e a aplicación execútase cun perfil de proba que recrea o esquema da base de datos en cada execución. Deste xeito verifícanse os fluxos completos da API: autenticación, xestión de viaxes e membros, gastos e liquidacións, eventos, documentos, chat e asistente de IA.

A cobertura de código mídese con **JaCoCo** [15]. As métricas exclúen o código xerado automaticamente a partir da especificación OpenAPI (os *DTO* e as interfaces **Api*), seguindo a práctica habitual de non contabilizar o código que non se escribe a man: incluílo só distorsionaría as cifras, xa que os seus métodos `equals`, `hashCode` e construtores nunca se exercitan. A Táboa 4.2 resume os resultados obtidos sobre o código propio na execución completa da batería.

Métrica	Cobertura
Liñas	92,6 %
Instrucións	91,4 %
Métodos	92,9 %
Clases	96,7 %
Ramas	85,0 %

Cadro 4.2: Cobertura de código do backend (JaCoCo).

A cobertura concéntrase na capa de servizos e nos fluxos principais da API, que é onde reside a lóxica de negocio, e supera o 90 % de liñas. As cifras lixeiramente máis baixas en ramas corresponden, en boa medida, a ramas defensivas e de manexo de erros pouco frecuentes. Ademais, a construción e a execución das probas intégranse nun fluxo de integración continua (GitHub Actions), e o código sométese a análise estática.

4.3. Probas automatizadas do frontend

O frontend conta cunha batería de **77 probas automatizadas** en 8 suites, executadas con **Jest** mediante o preset oficial de Expo (`jest-expo`). Para as probas de compoñentes e *hooks* utilízase `@testing-library/react-native`, que promove probas centradas no comportamento observable polo usuario en lugar de detalles de implementación internos.

As probas organizáronse segundo dúas orientacións complementarias:

- **Probas de contrato e lóxica pura:** cubren módulos sen dependencias de React Native, como a clase de erros (`AppError`), as claves de caché de React Query (`queryKeys`), as funcións auxiliares de documentos (`documentCards.helpers`) e as funcións de persistencia de sesión (`lastTrip`). Son especialmente robustas fronte a cambios de interface: só fallan se muda a lóxica de negocio.
- **Probas de comportamento e interacción:** cubren o *hook* de accesibilidade (`useReducedMotion`), a función de subida de ficheiros (`upload`), o compoñente

de texto temático (`ThemedText`) e o control de segmentos (`SegmentedControl`). Verifican o comportamento observable: que ao premer un segmento se invoca o *callback* co valor correcto, ou que un código de estado HTTP de erro rexeita a promesa.

A cobertura mídese con Jest sobre os **módulos directamente exercitados** polas probas. Por defecto, Jest só contabiliza os ficheiros que se importan durante a execución, de xeito que as pantallas da aplicación, os hooks de datos e os contextos de autenticación —que non forman parte desta batería— non aparecen na análise, nin sequera con 0 %. Polo tanto, as cifras da Táboa 4.3 reflicten a calidade da cobertura dentro dos módulos elixidos, non a cobertura global do frontend; o resto do código non dispón aínda de cobertura automatizada.

Módulo	Instrucións	Ramas	Funcións	Liñas
<code>lastTrip.ts</code>	100 %	100 %	100 %	100 %
<code>upload.ts</code>	100 %	100 %	100 %	100 %
<code>useReducedMotion.ts</code>	100 %	50 %	100 %	100 %
<code>ThemedText.tsx</code>	100 %	86 %	100 %	100 %
<code>documentCards.helpers.ts</code>	92 %	80 %	100 %	91 %
<code>queryKeys.ts</code>	92 %	100 %	88 %	92 %
<code>SegmentedControl.tsx</code>	80 %	41 %	83 %	77 %
<code>AppError.ts</code>	22 %	25 %	50 %	22 %
Media	87 %	64 %	91 %	86 %

Cadro 4.3: Cobertura de código do frontend (Jest) sobre os módulos probados.

A cobertura baixa de `AppError.ts` (22 % de instrucións) débese a que o ficheiro contén unha función privada non exportada (`mapHttpStatusToErrorCode`) que as probas non poden invocar directamente; a API pública do módulo —a clase e a enumeración— está completamente cuberta. As cifras de `SegmentedControl.tsx` reflicten que o código de animación e o manejador de *layout* non se activan no entorno de proba, onde o compoñente non realiza medicións reais de anchura. A rama non cuberta de `useReducedMotion.ts` correspóndese co caso de carreira no que o compoñente se desmonta xusto antes de que a promesa de `AccessibilityInfo` resolva, o que resulta difícil de reproducir de forma determinista nun test.

4.4. Validación funcional do sistema

Para verificar o comportamento de extremo a extremo executáronse manualmente sobre a aplicación os escenarios de uso descritos na Táboa 4.4, que cobren a totalidade dos requisitos funcionais. Cada escenario comprende a secuencia completa de accións do usuario e a comprobación do resultado esperado.

ID	RF	Escenario verificado
CP01	RF01, RF02	Rexistro, inicio e peche de sesión, e renovación de token.
CP02	RF03–RF05	Crear viaxe, convidar e aceptar membros (e por código QR).
CP03	RF06, RF07	Rexistrar gastos, consultar balances e liquidar débedas.
CP04	RF08	Crear, consultar e eliminar eventos do calendario.
CP05	RF09	Subir, previsualizar e eliminar documentos.
CP06	RF10	Enviar e recibir mensaxes no chat de grupo en tempo real.
CP07	RF11	Conversar co asistente de IA e recuperar o historial.
CP08	RF12, RF13	Xestionar amizades e convidar amigos a unha viaxe.
CP09	RF14, RF15	Editar o perfil e eliminar a conta con confirmación.

Cadro 4.4: Escenarios de validación funcional e a súa trazabilidade cos requisitos.

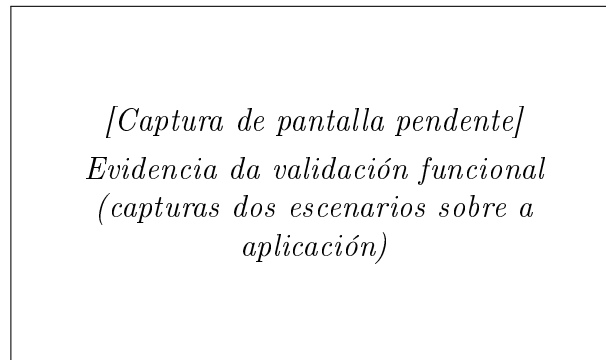


Figura 4.1: Evidencia da validación funcional (capturas dos escenarios sobre a aplicación)

4.5. Avaliación de usabilidade con usuarios reais

Para complementar a validación técnica cunha perspectiva centrada na experiencia de uso, realizouse unha avaliación de usabilidade mediante o **protocolo de pensar en voz alta** (*think-aloud*) [16]. Nesta técnica, os participantes verbalizan os seus pensamentos mentres interactúan libremente co sistema, o que permite ao avaliador identificar tanto obstáculos de navegación como erros de comprensión que os escenarios de validación estruturados non detectan.

As sesións realizáronse de forma moderada con **15 participantes** en total. Non se definiron tarefas pechadas: pediúselle a cada participante que explorara a aplicación como se a fose a usar no seu día a día, interviñendo o avaliador só para solicitar que verbalizasen o que estaban a pensar. Anotábanse os problemas observados e, ao rematar a sesión, recollíanse as impresións xerais do participante.

Os problemas identificados priorizábanse segundo dous criterios: (1) **frecuencia**, cando varios participantes atopaban o mesmo obstáculo; e (2) **severidade**, cando un único participante atopaba un bloqueo evidente que impedía completar un fluxo. Este dobre criterio é coherente coa práctica habitual da enxeñaría de usabilidade para distinguir incidencias menores de problemas críticos [16]. O proceso continuou de forma

iterativa, aplicando correccións entre sesións, ata alcanzar a **saturación**: o punto no que as sesións deixaron de producir problemas novos significativos.

Entre os aspectos corrixidos como resultado desta avaliación destacan:

- Acceso directo aos documentos do día seleccionado desde o calendario, que anteriormente requiría navegar a unha pantalla separada.
- Eliminación do diálogo de confirmación para volver ao menú principal, percibido como un paso innecesario que interrompía o fluxo de navegación.
- Ordenación das viaxes por data de creación, xa que os participantes esperaban atopar a viaxe máis recente en primeiro lugar.
- Incorporación dun mapa interactivo no modal de creación de eventos, para facilitar a selección da localización sen abandonar a pantalla.
- Engadido de categorías aos gastos, que varios participantes botaron en falta para organizar e filtrar as partidas dunha viaxe.

Ao final das sesións, os participantes completaban todos os fluxos principais sen atopar obstáculos relevantes e valoraban positivamente a claridade da navegación e a consistencia visual da aplicación.

4.6. Resultados acadados

A avaliación de usabilidade con 15 participantes mediante o protocolo de pensar en voz alta alcanzou a saturación sen detectar problemas críticos pendentes, confirmando que os fluxos principais son navegables e comprensibles para os usuarios obxectivo.

A totalidade das **487 probas automatizadas pasan correctamente**: 410 do backend (JUnit 5) e 77 do frontend (Jest). Os nove escenarios de validación funcional completáronse co resultado esperado, polo que se verifica o cumprimento de todos os requisitos funcionais. A cobertura do backend supera o 90 % de liñas sobre o código propio; no frontend, os 8 módulos directamente exercitados polas probas acadan de media un 86 % de liñas e un 91 % de funcións, aínda que estes valores non reflicten a cobertura global do frontend.

Capítulo 5

Conclusións e Posibles Ampliacións

5.1. Grao de cumprimento dos obxectivos

O proxecto acadou os obxectivos formulados na introdución. Telaria centraliza nunha única aplicación a xestión integral dunha viaxe en grupo, substituíndo o conxunto disperso de ferramentas que se empregan habitualmente. A Táboa 5.1 resume o grao de cumprimento de cada obxectivo.

Obxectivo	Estado	Observacións
Gastos compartidos e liquidacións	Cumprido	Cálculo no servidor con minimización de transferencias (RF06, RF07).
Calendario de eventos con localización	Cumprido	Eventos con data, duración e localización xeográfica (RF08).
Documentos compartidos	Cumprido	Subida directa ao almacén con miniaturas (RF09).
Chat de grupo	Cumprido	Mensaxería en tempo real mediante SSE (RF10).
Asistente de IA por viaxe	Cumprido	Chatbot con historial persistente sobre Ollama (RF11).
Rede de amizades (complementaria)	Cumprido	Solicitudes, lista de amigos e convite áxil (RF12, RF13).
Xestión de conta	Cumprido	Perfil, contrasinal, avatar e eliminación con confirmación (RF14, RF15).
Arquitectura API-first	Cumprido	Contrato OpenAPI con xeración de código en ambos os extremos.
Multiplataforma iOS/Android	Cumprido	Única base de código con React Native e Expo.

Cadro 5.1: Grao de cumprimento dos obxectivos.

5.2. Valoración técnica

A adopción dunha arquitectura *API-first* demostrou ser un acerto: ter o contrato OpenAPI como fonte de verdade permitiu xerar automaticamente os tipos do cliente e as interfaces do servidor, mantendo ambos os extremos sincronizados e detectando incoherencias en tempo de compilación. O uso de *Server-Sent Events* para o chat e o asistente de IA, e o das URLs prefirmadas para os ficheiros, permitiron implementar funcionalidades esixentes sen sobrecargar o servidor. A execución local do modelo de linguaxe con Ollama evitou a dependencia de servizos externos de pago. Finalmente, a batería de probas automatizadas do backend proporcionou unha rede de seguridade que facilitou a evolución continua do código.

Entre as dificultades atopadas destacan a integración de Testcontainers cun entorno de contedores sen daemon (Podman), a coordinación dos fluxos asíncronos de *streaming* e a xestión coherente da subida de ficheiros en varios pasos.

5.3. Posibles ampliacións

O sistema deixa aberto un conxunto de liñas de mellora:

- **Notificacións push** para invitacións, mensaxes novas e recordatorios de eventos.
- **Preparación para produción:** externalizar os segredos de configuración (actualmente en ficheiros de configuración), habilitar HTTPS/TLS e facer configurable a dirección do servidor no cliente.
- **Probas automatizadas no frontend** (por exemplo, con Jest e React Native Testing Library), que complementarían a sólida cobertura do backend.
- **Migracións de base de datos explícitas** (Flyway ou Liquibase) en lugar da xeración automática de esquema, de cara a un control de versións do esquema en produción.
- **Visualización de mapas** para a localización dos eventos, aproveitando as coordenadas xa almacenadas.
- **Modo sen conexión** con sincronización posterior, para mellorar a experiencia en destinos con conectividade limitada.
- **Accesibilidade:** estender o límite de escala da tipografía a todos os textos da interface, sobre a base do etiquetado para lectores de pantalla, o axuste de contraste WCAG AA, o respecto do movemento reducido, as áreas de pulsación ampliadas e as pistas de accesibilidade xa implementados (ver a subsección de deseño da interface de usuario).

Apéndice A

Manual técnico

Este manual recolle a información necesaria para instalar, executar, probar e modificar Telaria. Diríxese a persoas que vaian continuar o desenvolvemento do sistema.

A.1. Requisitos do entorno

- **JDK 25** (para o backend; a construción realízase co *wrapper* de Gradle).
- **Node.js** con **npm** (para o frontend e a xeración de tipos).
- **Podman** con soporte de `podman compose` (para os servizos de apoio).
- **Git**.

A.2. Estrutura do repositorio

- `backend/` — API REST en Spring Boot (Gradle).
- `frontend/` — aplicación móbil en React Native + Expo.
- `memoria/` — esta documentación (LaTeX).

A.3. Servizos de apoio (Podman)

Os servizos auxiliares (PostgreSQL, MinIO e Ollama) defínense en `backend/compose.yaml`. O xeito máis sinxelo de poñelos en marcha, descargar o modelo de linguaxe e arrincar o backend é o *script* de inicio:

```
cd backend
./start.sh
```

Alternativamente, pódense levantar só os contedores:

```
cd backend
podman compose up -d # engadir --profile gpu se hai GPU dispoñible
```

Portos por defecto: PostgreSQL 5432, MinIO 9000/9001 (API/consola), Ollama 11434.

A.4. Execución do backend

```
cd backend
./gradlew bootRun
```

O servidor escoita no porto 8082. A configuración (base de datos, *keystore* de sinatura JWT, MinIO e tarefas programadas) atópase en `src/main/resources/application.yaml`. A documentación interactiva da API queda dispoñible en `/swagger-ui.html`.

A.5. Execución do frontend

```
cd frontend
npm install
npm start # inicia o servidor de desenvolvemento de Expo
```

A dirección do backend configúrase na constante `BASE_URL` de `frontend/constants/constants.tsx`, que debe apuntar ao enderezo accesible do servidor desde o dispositivo. A aplicación pode abrirse en Expo Go ou nun emulador (`npm run android` / `npm run ios`).

A.6. Contrato OpenAPI e xeración de código

A API especificase en `backend/src/main/resources/openapi/`, cun documento principal (`api.yaml`) que referencia ficheiros de rutas e esquemas. A partir del xérase código:

```
# Backend: interfaces de controladores e DTO
cd backend && ./gradlew openApiGenerate

# Frontend: tipos TypeScript
cd frontend && npm run generate:types
```

Ao editar ficheiros referenciados con `$ref` (rutas ou esquemas), pode ser preciso forzar a rexeneración, xa que a tarefa de Gradle só observa cambios en `api.yaml`.

A.7. Execución das probas

As probas de integración (E2E) empregan Testcontainers, que require un *socket* compatible con Docker. Con Podman *rootless*, débese indicar a súa ruta:

```
cd backend
export DOCKER_HOST=unix:///run/user/$(id -u)/podman/podman.sock
export TESTCONTAINERS_RYUK_DISABLED=true
./gradlew test
```

Ao rematar, o informe de cobertura JaCoCo xérase en `backend/build/reports/jacoco/test/html/index.html`. A análise estática SpotBugs está dispoñible mediante a tarefa correspondente de Gradle.

A.8. Configuración e segredos

A configuración do backend (credenciais da base de datos, contrasinal do *keystore* JWT, claves de acceso a MinIO e parámetros das tarefas programadas) reside en `application.yaml`. Para un despregamento en produción recoméndase externalizar estes valores mediante variables de entorno e non incluílos no control de versións.

A.9. Integración continua

O repositorio inclúe fluxos de traballo de GitHub Actions que executan a construción e as probas do backend, así como unha análise de seguridade do código (CodeQL).

Apéndice B

Manual de usuario

Este manual é autocontido e está dirixido ás persoas que utilicen Telaria. Describe a instalación e o uso de todas as funcionalidades. En cada sección inclúense capturas de pantalla representativas da aplicación, que poden axudar ao usuario a familiarizarse coas distintas pantallas e fluxos de interacción. As capturas de pantalla foron tomadas empregando o modo claro da aplicación para unha maior claridade visual, pero a funcionalidade é a mesma no modo escuro.

B.1. Instalación e acceso

Telaria é unha aplicación móbil para iOS e Android. Durante o desenvolvemento pode executarse a través de Expo Go ou instalando unha compilación da aplicación. Ao abrila por primeira vez, o usuario debe rexistrarse ou iniciar sesión.

B.2. Rexistro e inicio de sesión

Na pantalla inicial pódese crear unha conta nova indicando nome de usuario, correo electrónico e contrasinal (con confirmación), ou iniciar sesión cunha conta existente. A sesión mantense aberta entre usos.

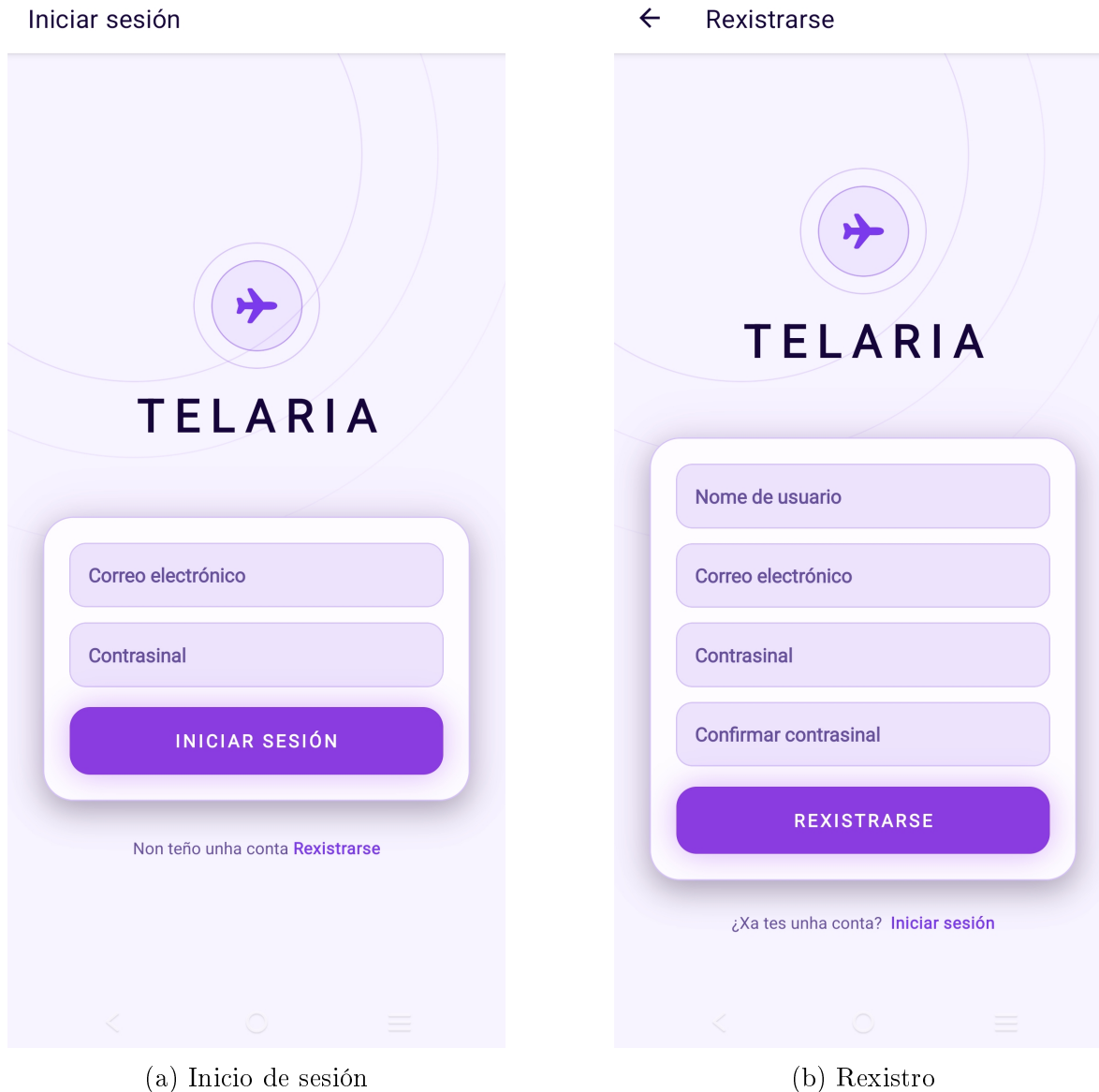


Figura B.1: Pantallas de rexistro e inicio de sesión

B.3. Pantalla principal: as miñas viaxes

Tras iniciar sesión, móstrase a lista de viaxes do usuario. Desde aquí pódese acceder a unha viaxe, crear unha nova ou unirse a unha existente, así como abrir o menú lateral para acceder a amigos, invitacións, perfil e axustes.



Figura B.2: Pantalla principal, coas viaxes do usuario

B.4. Crear e unirse a viaxes

Para crear unha viaxe abonda con indicar o seu nome; opcionalmente pódense convidar amigos no momento da creación. Para unirse a unha viaxe existente pódese empregar unha invitación recibida, enviar unha solicitude de unión ou escanear un código QR.

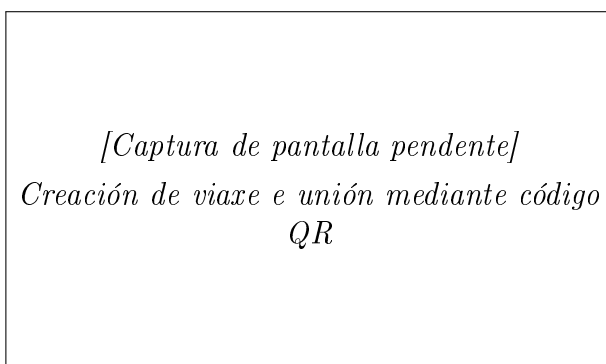


Figura B.3: Creación de viaxe e unión mediante código QR

B.5. Xestión de membros

Dentro dunha viaxe, calquera membro pode convidar outros usuarios e xestionar as solicitudes de unión pendentes. Non existen roles: todos os membros teñen as mesmas capacidades.

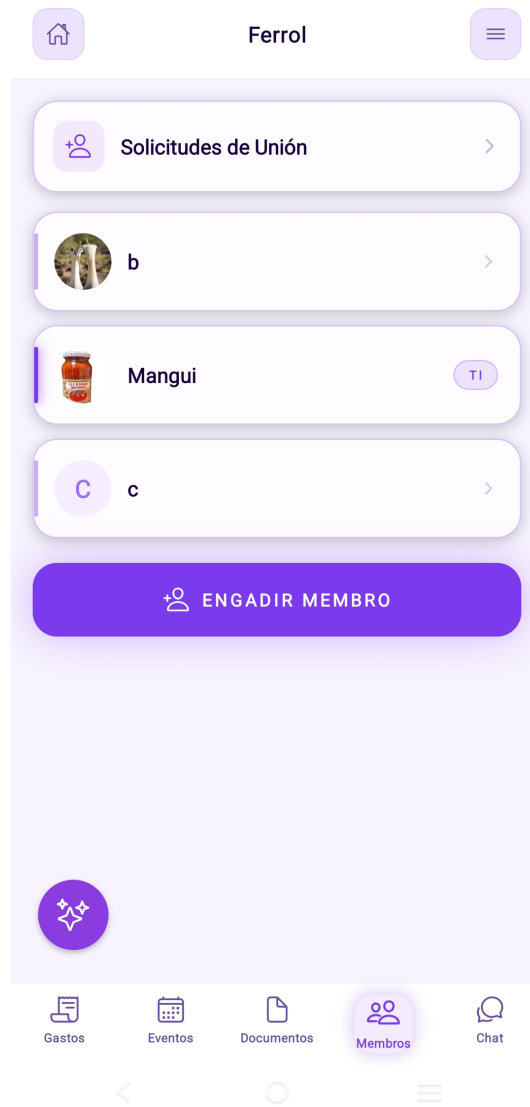
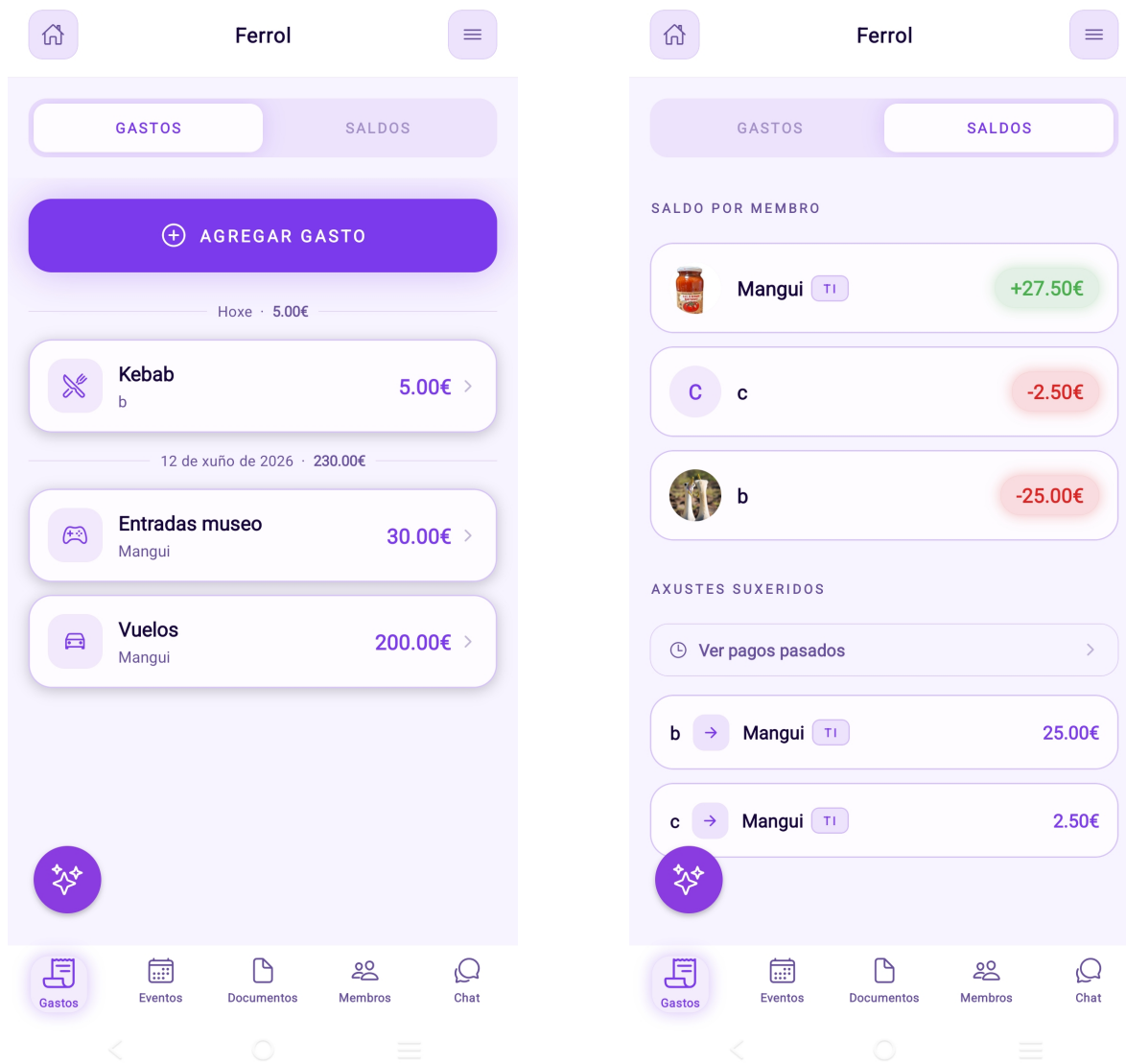


Figura B.4: Membros da viaxe e solicitudes pendentes

B.6. Gastos e liquidacións

No apartado de gastos rexístranse os pagamentos indicando importe, concepto, categoría, quen pagou e entre quen se reparte. A aplicación calcula automaticamente os balances (quen debe a quen) e propón as liquidacións que minimizan o número de transferencias. As débedas saldadas rexístranse no histórico de liquidacións.



(a) Pestaña de gastos

(b) Pestaña de liquidacións

Figura B.5: Lista de gastos e liquidacións

B.7. Eventos e calendario

O calendario da viaxe permite crear eventos con nome, data e hora de inicio, duración e unha localización opcional. Os eventos amósanse organizados por data.

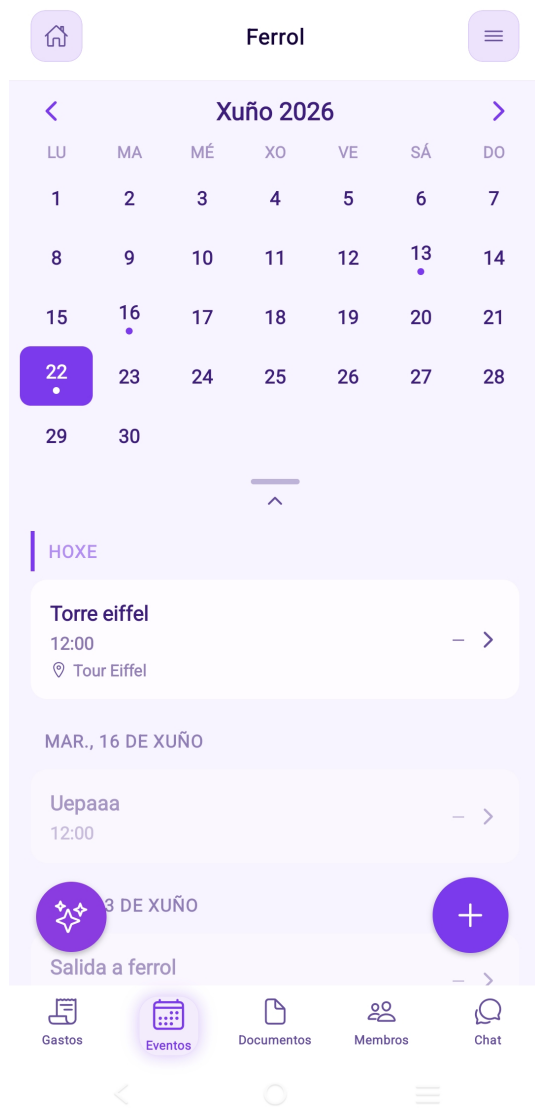


Figura B.6: Calendario e creación de eventos

B.8. Documentos compartidos

Os membros poden subir documentos relevantes para a viaxe (reservas, billetes, etc.) e consultalos ou eliminalos. Os documentos de imaxe amosan unha miniatura de previsualización.

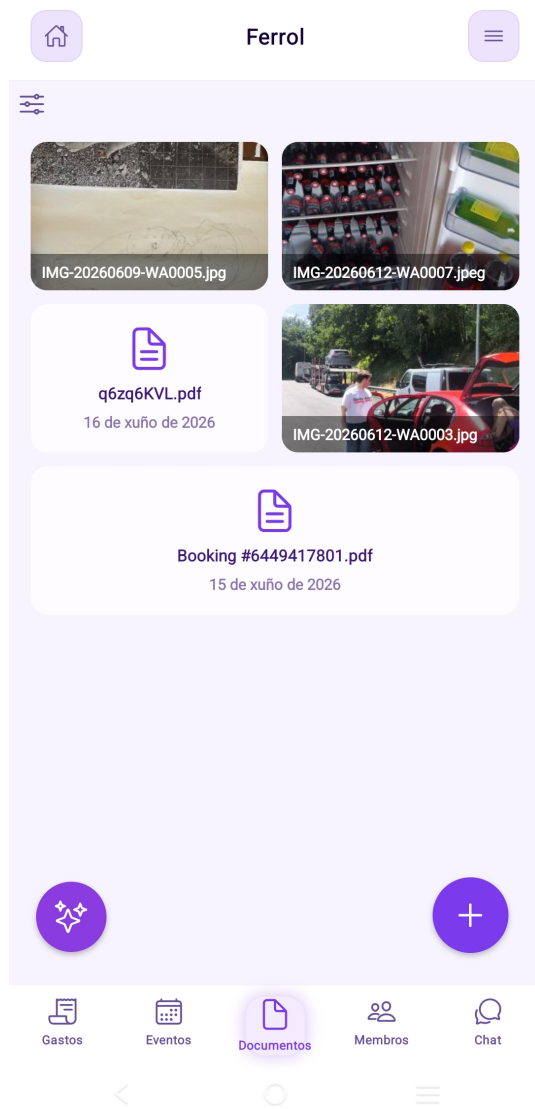


Figura B.7: Listaxe de documentos compartidos

B.9. Chat de grupo

Cada viaxe dispón dun chat común no que os membros intercambian mensaxes en tempo real.

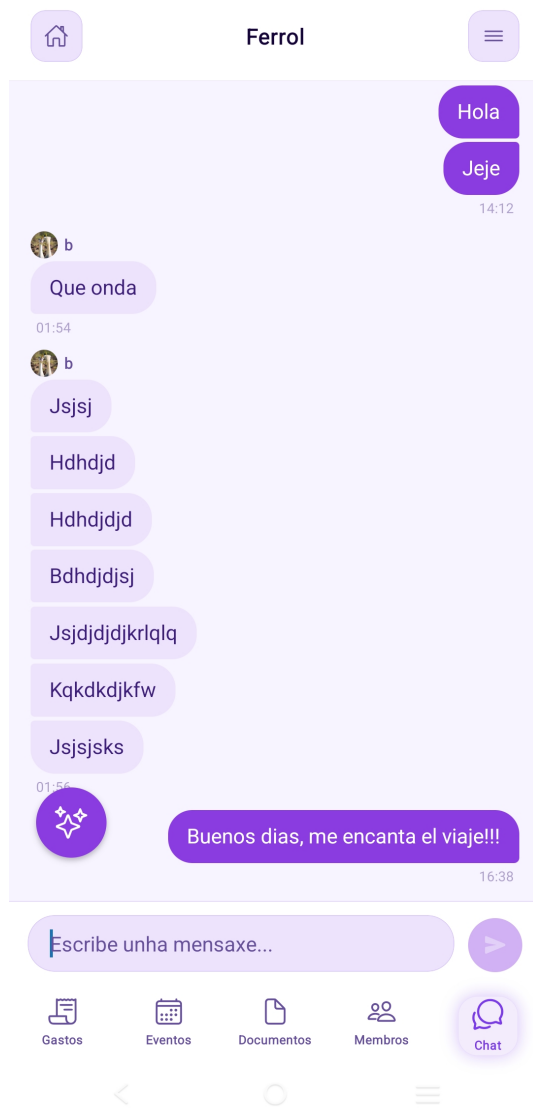


Figura B.8: Chat de grupo da viaxe

B.10. Asistente de intelixencia artificial

Cada viaxe inclúe un asistente conversacional que responde preguntas tendo en conta o contexto da viaxe (eventos e gastos). A conversa garda un historial persistente.

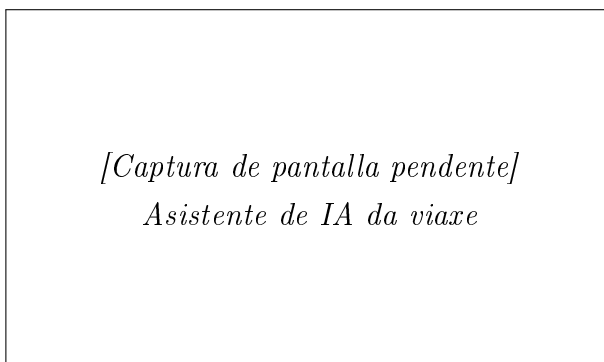


Figura B.9: Asistente de IA da viaxe

B.11. Amizades

A sección de amigos permite enviar e xestionar solicitudes de amizade (por identificador ou código QR), consultar a lista de amigos e eliminar amizades. Os amigos poden convidarse ás viaxes de forma máis áxil.

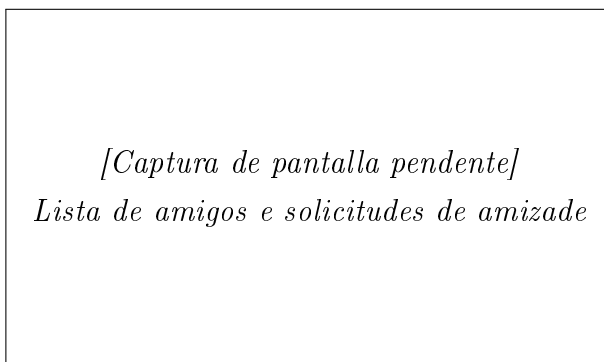


Figura B.10: Lista de amigos e solicitudes de amizade

B.12. Invitacións

Na caixa de entrada de invitacións agrúpanse as invitacións a viaxes e as solicitudes de amizade recibidas. Desde aquí pódense aceptar ou rexeitar cada unha de forma individual.

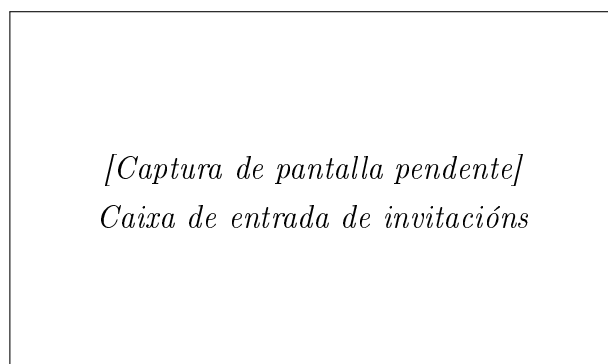


Figura B.11: Caixa de entrada de invitacións

B.13. Perfil

No perfil pódese consultar e actualizar o nome de usuario e o correo electrónico (o cambio de correo require o contrasinal), cambiar o contrasinal e establecer un avatar.



Figura B.12: Perfil de usuario

B.14. Axustes

Na pantalla de axustes configúrase o idioma (galego, castelán ou inglés), o tema (claro ou escuro) e o aforro de datos.



Figura B.13: Pantalla de axustes

B.15. Eliminación da conta

Desde os axustes, o usuario pode eliminar a súa conta. A operación require a confirmación do contrasinal e elimina os datos persoais asociados.

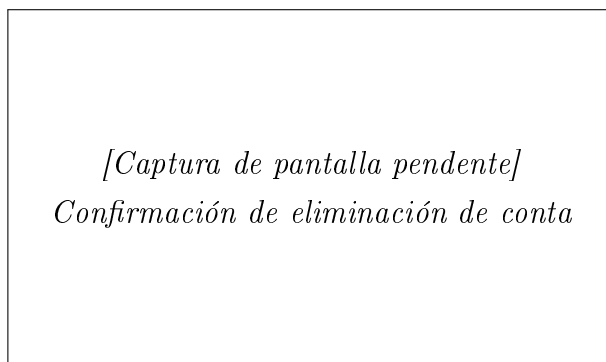


Figura B.14: Confirmación de eliminación de conta

B.16. Mensaxes de erro comúns

- **Credenciais incorrectas:** o correo ou o contrasinal non son válidos no inicio de sesión.
- **Correo xa rexistrado:** existe xa unha conta con ese correo ao rexistrarse.
- **Acción xa realizada:** por exemplo, o usuario xa é membro da viaxe ou xa foi convidado.
- **Sen conexión:** a aplicación require conexión á rede; comprobe a conectividade e que o servidor estea accesible.

Apéndice C

Licenza

Bibliografia

- [1] Meta Platforms, Inc. *React Native. A framework for building native apps using React*. <https://reactnative.dev>
- [2] Expo. *Expo Documentation*. <https://docs.expo.dev>
- [3] TanStack. *TanStack Query (React Query)*. <https://tanstack.com/query>
- [4] Broadcom (VMware Tanzu). *Spring Boot Reference Documentation*. <https://spring.io/projects/spring-boot>
- [5] Broadcom (VMware Tanzu). *Spring Security Reference Documentation*. <https://spring.io/projects/spring-security>
- [6] OpenAPI Initiative. *OpenAPI Specification*. <https://spec.openapis.org>
- [7] The PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- [8] M. Jones, J. Bradley e N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [9] M. Nottingham e E. Wilde. *Problem Details for HTTP APIs*. RFC 7807, IETF, 2016. <https://www.rfc-editor.org/rfc/rfc7807>
- [10] Ollama. *Ollama. Get up and running with large language models locally*. <https://ollama.com>
- [11] MinIO, Inc. *MinIO Object Storage Documentation*. <https://min.io>
- [12] N. Provos e D. Mazières. *A Future-Adaptable Password Scheme*. Proceedings of the USENIX Annual Technical Conference, 1999.
- [13] Red Hat, Inc. *Podman. A tool for managing OCI containers and pods*. <https://podman.io>
- [14] AtomicJar / Docker, Inc. *Testcontainers for Java*. <https://java.testcontainers.org>
- [15] *JaCoCo Java Code Coverage Library*. <https://www.jacoco.org/jacoco/>
- [16] J. Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.
- [17] Tricount (Bunq). *Tricount. Split group expenses*. <https://www.tricount.com>

- [18] WhatsApp LLC. *WhatsApp Messenger*. <https://www.whatsapp.com>
- [19] Google LLC. *Google Calendar*. <https://calendar.google.com>
- [20] Wanderlog. *Wanderlog. Travel itinerary and trip planner*. <https://wanderlog.com>
- [21] TravelPerk. *TravelPerk. Business travel management*. <https://www.travelperk.com>